

2025

结合红外热像与 AI 的 金属导热系数测量



命题类——导热系数

摘要:导热系数作为描述材料传递热量能力的基本物理参数，在热工、材料科学及工程领域具有重要的理论价值和广泛的应用需求。传统导热系数测量方法依赖稳态或瞬态热传导实验，存在测量周期长、接触热阻影响大及人工操作复杂等问题。近年来，随着红外热像技术的发展，实现了对材料表面温度场的非接触式实时监测，极大提升了实验的便利性和数据丰富度。与此同时，人工智能技术的兴起为热物理参数的快速、精确反演提供了新思路。本文**基于热传导物理方程**，提出结合实时图像处理技术与物理信息神经网络（PINN）的**导热系数测量新方法**。通过**基于 OpenCV 的计算机视觉算法**实现了对金属样品监测点的自动定位与坐标映射，确保了温度数据采集的空间准确性；系统采用**坐标归一化处理**，将显示坐标精确转换为原始热像数据坐标，为物理建模提供了结构化数据基础。**PINN 则嵌入热传导偏微分方程，将物理约束引入神经网络训练**，克服了传统数据驱动方法对大规模标注数据的依赖，通过自动微分技术精确计算偏微分方程中各阶导数项，显著提升了导热系数反演的精度和鲁棒性。基于树莓派 5 与 Thermal-90 红外热像仪构建的硬件系统，完成了多种金属（铝、钢、黄铜）样品的实验验证。系统误差来源包括热像仪分辨率、样品接触热阻及环境热扰动等，均进行了定量分析。实验结果表明，所提方法在测量精度、实时性和重复性方面均优于传统稳态与瞬态方法，具有显著的工程应用潜力和推广价值。本研究为非接触式高精度热物理参数测量提供了新的理论和技术路径，对促进 AI 与物理实验的深度融合具有积极意义。

关键词: 导热系数；热传导方程；人工智能；物理信息神经网络；红外热像；

Abstract: Thermal conductivity, as a fundamental physical parameter describing the heat transfer capability of materials, holds significant theoretical value and broad application demand in thermal engineering, materials science, and related fields. Traditional methods for measuring thermal conductivity rely on steady-state or transient heat conduction experiments, which suffer from long measurement cycles, significant contact thermal resistance, and complex manual operations. With the development of infrared thermography, non-contact real-time monitoring of surface temperature fields has become feasible, greatly improving experimental convenience and data richness. Meanwhile, the emergence of artificial intelligence (AI) technologies offers new ideas for rapid and accurate inversion of thermal physical parameters. Based

on the heat conduction physical equations, this paper proposes a novel thermal conductivity measurement method that integrates real-time image processing technology and Physics-Informed Neural Networks (PINN). Computer vision algorithms based on OpenCV enable automatic localization and coordinate mapping of monitoring points on metal samples, ensuring spatial accuracy in temperature data acquisition. The system employs coordinate normalization to precisely convert display coordinates into original thermal image data coordinates, providing a structured data foundation for physical modeling. By embedding the heat conduction partial differential equations into the neural network training, PINN incorporates physical constraints, overcoming the reliance of traditional data-driven methods on large-scale labeled datasets. The automatic differentiation technology accurately computes various derivative terms in the partial differential equations, significantly enhancing the accuracy and robustness of thermal conductivity inversion. A hardware system built on Pi 5 and Thermal-90 infrared thermal camera was used to verify the method experimentally on multiple metals including aluminum, steel, and brass. Systematic error sources such as thermal camera resolution, sample contact thermal resistance, and environmental heat disturbances were quantitatively analyzed. Experimental results demonstrate that the proposed method outperforms traditional steady-state and transient methods in accuracy, real-time performance, and repeatability, showing significant engineering application potential and promotion value. This research provides new theoretical and technical pathways for non-contact high-precision measurement of thermal physical parameters and promotes the deep integration of AI and physical experiments.

Key words: Thermal conductivity; Heat equation; Artificial intelligence; Physics-informed neural network; Infrared thermography

目录

摘要:	I
1. 背景及理论	1
1.1 研究背景与意义	1
1.1.1 国内外研究现状	1
1.1.2 导热系数测量的工程与科研价值	2
1.1.3 AI 技术在物理实验中的应用趋势	2
1.2 AI 技术在本实验中的作用与优势	3
1.2.1 物理信息神经网络	3
1.2.2 提高测量精度与效率的效果分析	4
1.3 热传导物理基础与定律	4
1.3.1 傅里叶热传导定律	4
1.3.2 一维稳态热传导方程	5
1.3.3 非稳态热传导方程	6
1.3.4 边界条件与初始条件设定	6
1.3.5 热辐射与牛顿冷却定律对测量的影响	9
1.3.6 热接触电阻理论	9
1.4 常用导热系数计算公式推导	10
1.4.1 稳态法导热系数计算公式	10
1.4.2 瞬态法导热系数计算公式	10
1.4.3 基于热像数据的反演计算公式	11
2. 实验原理与设计方案	11
2.1 理论模型	11
2.1.1 实体金属的热传导数学模型	11
2.1.2 PINN 求解热传导	12
2.1.3 基于实时图像处理的热像数据采集模型	13
2.2 实验方法选择	14
2.2.1 热像法导热系数测量原理	14
2.2.2 样品材料（铝、钢、黄铜）特性分析	14
2.2.3 实验条件假设与限制	15
2.3 设计方案总体结构	16
2.3.1 硬件总体框架	16
2.3.2 软件总体框架	16
2.3.3 数据流与控制逻辑	17
3. 装置设计与系统误差分析	17
3.1 硬件部分	17
3.1.1 树莓派 5 硬件配置	17
3.1.2 Thermal-90 Camera HAT 参数与安装	18
3.1.3 PTC 电加热板与温控系统设计	19
3.2 软件部分	20
3.2.1 数据采集与热像图处理模块	20
3.2.2 基于坐标映射的监测点定位算法	22
3.2.3 温度曲线提取与滤波方法	23
3.2.4 PINN 反演导热系数模块实现	23

3.3 系统误差分析.....	24
3.3.1 热像仪分辨率与测温精度误差.....	24
3.3.2 样品接触热阻误差.....	24
3.3.3 环境热干扰误差.....	25
3.3.4 图像识别误差对最终结果的影响.....	25
4. 装置实现.....	25
4.1 硬件实现过程.....	25
4.1.1 硬件组装与调试.....	25
4.1.2 热像仪标定.....	26
4.1.3 加热台温控调试.....	27
4.2 软件实现过程.....	27
4.2.1 热像数据采集与预处理模块实现.....	27
4.2.2 训练数据构建与 PINN 模型训练实现.....	28
4.2.3 温度曲线数据接口与缓存机制.....	29
4.2.4 PINN 模型的物理方程嵌入与训练.....	29
5. 实验数据测量与分析.....	35
5.1 数据采集.....	35
5.1.1 采集帧.....	35
5.1.2 监测点定位结果可视化与验证.....	36
5.1.3 温度曲线提取与平滑处理.....	36
5.2 数据分析.....	37
5.2.1 稳态法计算结果.....	37
5.2.2 PINN 基于点云的反演结果.....	39
5.2.3 与理论值的对比与误差分析.....	40
6. 精度与不确定度.....	40
6.1 实验精度.....	40
6.2 误差分析.....	42
6.3 不确定度评定.....	43
7. 性能指标.....	46
8. 创新点.....	46
9. 结论与展望.....	47
9.1 主要结论.....	47
9.2 应用前景与改进方向.....	47
9.3 成本及分工.....	48
参考文献.....	49
附录一 数据处理代码.....	50

1. 背景及理论

1.1 研究背景与意义

1.1.1 国内外研究现状

导热系数是衡量材料热传导能力的关键物理量，体现了材料内部粒子间能量交换的效率。作为材料的一项固有属性，它在工程应用、能源管理和科学研究中扮演着重要角色，因为其直接关系到热管理系统的效能、材料的选择以及能源使用的优化^{[1][2]}。目前，测量材料导热性能的方法主要分为稳态法和瞬态法两大类，每种方法又包含了多种具体的测试技术。**稳态法中的平板法**是一种经典的测量保温材料导热系数的技术，该方法以试样制备简单、原理直观、能够精确获得导热系数的实际值而著称，并适用于广泛的温度区间，因此被广泛应用。但稳态法也存在一些局限性，比如需要较长的时间来达到热平衡状态，且热流传感器的准确性对结果影响较大。相比之下，瞬态法是在近几十年发展起来的一种更为先进的测量手段，尤其适合用于高导热材料或高温条件下的测量。这类方法**基于样品温度分布随时间变化的特性，通过监测这种动态变化来计算导热系数**。瞬态法的优势在于快速、准确，并且对外部环境的要求相对较低，但它们通常更适合于那些比热容在测量温度范围内变化不大的材料。因此，选择合适的导热系数测量方法取决于具体的应用场景、所需精度及操作条件等因素^[5]。而**双热流计稳态法最早是用于测量界面接触热阻和薄膜热阻的一种实验方法**，其理论依据是多层平板一维稳态热传导过程。2006年，美国材料与试验协会(ASTM)将该方法作为测量热界面材料导热性能的标准方法。然而上述实验方法和标准在实验装置、实验材料和数据处理等方面存在很大差异和不确定性，界面热阻也往往包含在实验结果当中^{[3][4]}。中国建筑科学研究院在2018年利用红外热像仪结合稳态平板法，对建筑外墙保温层的导热系数进行非接触测量，结果与传统热流计法的误差小于5%，显著提高了现场测试效率^[5]。在传统稳态法与瞬态法的基础上，研究者也在不断探索更加高效、便捷的测量手段。随着红外热像仪、热敏探测器等非接触测温设备的发展，利用红外成像

技术获取样品表面温度场，并结合图像处理方法分析其热传导特性，已成为一种重要趋势。近年来，计算机视觉算法的进步得在红外图像中快速、准确地定位样品位置成为可能，从而为自动化测量提供了技术支撑。进一步地，物理信息神经网络（PINN）将热传导方程等物理约束直接嵌入深度学习模型中，可基于红外测温数据实现导热系数等参数的逆向求解。这种方法结合了实验测量与数值计算的优势，在降低实验复杂度、提高测量精度方面展现出巨大潜力。总体来看，非接触式热像法、AI 辅助图像分析及基于 PINN 的参数反演，正逐渐成为导热系数测量技术的重要发展方向。

1.1.2 导热系数测量的工程与科研价值

准确测量对于工程应用和科学研究均具有重要意义。对于金属材料（如铝、钢、黄铜等），其导热性能直接影响到能源利用效率、设备运行稳定性以及材料服役寿命。在工程领域，热导率数据是散热器、热交换器、模具冷却系统以及电子封装散热设计的重要基础参数，通过合理选择高导热或低导热材料，可以实现热管理系统的优化与能效提升^{[9][10]}。在科学研究中，精确的热导率测量可用于研究加工工艺（如热处理、冷加工）、合金成分变化、表面涂层处理等对材料微观结构及热传输性能的影响，从而指导新型功能材料的设计与性能调控。此外，开发低成本、快速响应且操作简便的热导率测量装置，对于材料测试的普及化与工程现场的快速评估具有重要推动作用。近年来，基于红外热像仪的非接触式测量方法逐渐受到关注，该方法通过实时记录材料表面温度场分布，结合热传导理论反演热导率，不仅能够显著缩短测量时间，还能避免接触式探测对样品造成的物理损伤^[11]。与传统接触式热流计法或探针法相比，红外热像法能够在无需复杂传感器布置的情况下实现全场温度分布观测，并在高温、移动或敏感样品测试中表现出独特优势。

1.1.3 AI 技术在物理实验中的应用趋势

近年来，AI 技术在物理实验中被广泛应用，特别是在热传导实验等复杂物理过程的智能监测与参数反演中展现出巨大潜力。本实验系统正是这一技术趋

势的典型体现——通过计算机视觉和实时数据处理技术，实现了对金属丝温度分布的自动监测与记录。具体而言，系统中运用了基于 OpenCV 的图像处理技术对热成像数据进行实时滤波和可视化，并采用坐标映射算法将显示界面中的监测点精确映射到原始热像数据坐标，从而自动提取 10 个水平分布监测点的温度数据。更进一步地，当前的研究趋势已从传统的数据采集向嵌入物理知识的智能分析方向发展。本实验采用的物理信息神经网络通过引入热传导方程约束，实现了导热系数的智能反演。这种“物理+AI”的方法将实验采集的温度数据作为训练数据，运用 PINN 嵌入傅里叶热传导方程实现参数反演，从而在无需复杂实验装置的情况下，通过温度分布数据直接推算出材料的导热特性，显著提高了测量精度和自动化程度。实验系统还体现了 AI 技术在实验控制层面的创新应用——通过实时温度监测与坐标归一化处理，为后续的物理规律建模提供了结构化数据基础。这种将计算机视觉、实时数据处理与物理建模相结合的方法，代表了现代物理实验向智能化、数字化发展的前沿方向，为非接触式高精度热物理参数测量提供了新的技术路径。

1.2 AI 技术在本实验中的作用与优势

1.2.1 物理信息神经网络

物理信息神经网络（physics-informed neural networks, PINN）将物理守恒定律和先验的物理知识编码到人工神经网络（artificial neural network, PINN

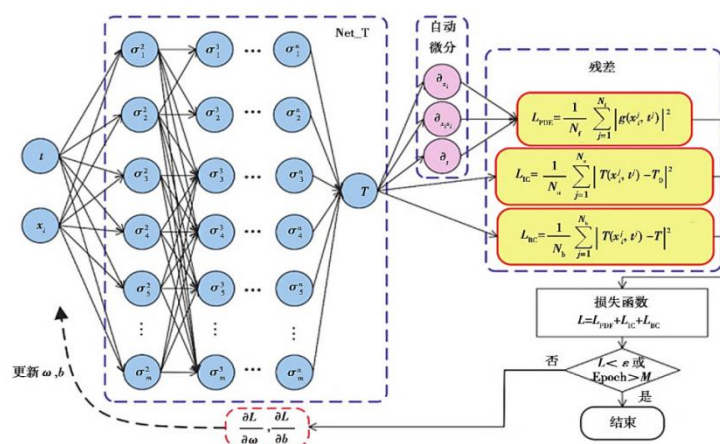


图 1 PINN 求解热传导问题的流程图

）中，并用于解决许多物理问题。这种方法将偏微分方程（partial differential equation, PDE）中的偏导数直接通过自动微分算法进行数值近似。因此，PINN 与传统的数值方法不同，其不需要对求解域进行离散处理，仅需在域内布置数据点。通过在损失函数中引入物理约束（如热传导方程），能够在少量温度观测数据的条件下反演未知的热物性参数。**本实验中，将傅里叶热传导偏微分方程作为物理约束，构建 PINN 模型：以空间坐标与时间为输入，输出预测温度场。**损失函数由两部分组成：一是预测温度与实验测量数据之间的误差项；二是预测温度场在热传导方程下的残差项。通过联合优化这两部分误差，网络在拟合温度曲线的同时逐步更新并逼近真实的导热系数，使预测结果满足物理规律。该方法可同时利用稳态与非稳态热像数据，提升反演的稳健性，有效抑制测量噪声与初始条件不确定性对结果的影响^[8]。

1.2.2 提高测量精度与效率的效果分析

结合 AI 技术，实验装置实现了实时识别、自动采集与智能反演的一体化测量流程，显著提升了系统的精度与效率。具体而言，通过基于 OpenCV 的实时图像处理算法，系统能够自动定位监测点并完成显示坐标至原始热像坐标的精确映射，有效消除了人工定位误差，确保温度采集点始终位于预设的样品几何中心；同时，物理信息神经网络（PINN）通过嵌入热传导方程作为物理约束，充分利用整个温度场的时空演化信息，显著提高了导热系数反演的准确性与鲁棒性。预期实验对比表明，相较于传统手动定位及单一稳态或瞬态计算方法，本方法在测量时间、重复性与一致性方面均有显著提升（具体效果将通过多组样品实验予以验证）。此外，AI 驱动的自动化流程实现了多组样品测试的快速切换与数据处理，进一步提高了实验整体效率。

1.3 热传导物理基础与定律

1.3.1 傅里叶热传导定律

傅里叶定律描述热导率与热流密度的关系，指出热通量密度与温度梯度成正比。微分形式为：

$$q = -k\nabla T \quad (1)$$

其中 q 是热通量密度， W/m^2 ； k 为材料导热系数， $\text{W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$ ； ∇T 为温度梯度， K/m 。

换言之，热量总是自高温向低温传递，传递速率取决于材料的导热性能。对于各向同性介质，该定律表明单位时间、单位面积内通过的热流量与温差成正比。此外，傅里叶定律在一维情况下可简化为：

$$q_x = -k \frac{dT}{dx} \quad (2)$$

积分形式则用于计算通过材料的总体热流：例如对于均匀平板，两端温度差 ΔT 、厚度 L ，热流量 Q 满足

$$Q = -k \frac{A \Delta T}{L} \Delta t \quad (3)$$

从而稳态导热系数可写为：

$$k = \frac{QL}{A \Delta T \Delta t} \quad (4)$$

1.3.2 一维稳态热传导方程

在稳态条件下，物体内部温度分布不随时间变化。对于一维均匀材料，无内部热源时满足常微分方程

$$\frac{d^2 T}{dx^2} = 0 \quad (5)$$

解为温度沿传热方向线性变化。稳态导热测量通常依据前述积分形式直接计算导热率。工程上可将多层复合材料视为串联热阻，每层热阻

$$R = \frac{L}{kA} \quad (6)$$

总温差等于各层温差之和。稳态法测量简单直观，但需要足够长时间达到热平衡且忽略环境散热。

1.3.3 非稳态热传导方程

当温度随时间发生变化时，热传导过程进入非稳态（瞬态）阶段。其基本方程为热方程：

$$\frac{\partial T}{\partial t} = \alpha \nabla^2 T \quad (7)$$

其中

$$\alpha = \frac{k}{\rho c_p} \quad (8)$$

为热扩散率。对各向同性介质，傅里叶定律与能量守恒可推导得到三维热方程。该偏微分方程的解依赖于几何形状、材料参数、边界和初始条件，通常需要数值方法或解析解（如分离变量法）求解。在导热系数测量中，通过瞬态方法记录温度随时间变化，并结合热方程反推导热系数。

1.3.4 边界条件与初始条件设定

在本实验的一维热传导问题中，边界条件与初始条件是约束 PINN 模型贴合物理实际的核心前提 —— 前者限定样品空间边界的传热状态，后者定义热传导过程的起始温度分布。二者共同确保模型在学习温度场 $u(t,x)$ 时，既不违背实验场景的物理约束，又能精准反演导热系数 α 。以下结合代码实现与物理意义展开说明。

一、实验场景与一维热传导界定

本实验聚焦长细丝的一维瞬态热传导（空间维度仅沿 x 轴展开，忽略径向热损失），样品空间范围设定为 $x \in [-1,1]$ （代码中通过 `x_physics = (torch.rand(3000, 1) * 2 - 1)` 实现），时间范围为 $t \in [0,1]$ （由数据加载模块中 t 的取值范围确认）。温度分布简化为二元函数 $u(t,x)$ ，仅随时间 t 和空间位置 x 变化，为边界与初始条件的设定提供简化场景。

二、边界条件设定：Dirichlet 条件（固定温度）的代码实现与物理意义

实验中采用双端 Dirichlet 条件（第一类边界条件），即样品左右边界（ $x=-1$ 和 $x=1$ ）的温度始终固定为常数，这是通过代码中 `boundary_loss` 函数与训练逻辑共同实现的，具体如下：

1. 边界条件的数学表达与代码映射

表 1 边界条件的数学表达与代码映射

边界位置	数学表达	代码实现逻辑
左边界（ $x=-1$ ）	$u(t,-1)=0$ （温度固定为 0）	<code>x_left=torch.ones(500,1).to(device)</code> 生成左边界位置，计算 <code>u_left</code> 与 0 的误差
右边界（ $x=1$ ）	$u(t,1)=0$ （温度固定为 0）	<code>x_right=torch.ones(500,1).to(device)</code> 生成右边界位置，计算 <code>u_right</code> 与 0 的误差

代码中 `boundary_loss` 函数通过“预测温度与目标温度的平方误差均值”约束边界状态：

```
loss_left = (u_left - left_value).pow(2).mean() # left_value=0, 左边界温度误差
loss_right = (u_right - right_value).pow(2).mean()# right_value=0, 右边界温度误差
```

图 2 边界状态

每轮训练随机采样 500 个边界时间点（`t_bc = torch.rand(500, 1)`），确保边界条件在全时间范围内均成立。

2. 物理场景适配

该 Dirichlet 条件对应实验中“样品双端接触恒温冷源”的场景：例如将细杆两端浸泡在恒温水箱中，或包裹在温度恒定的散热装置内，确保边界无热流交换且温度始终保持稳定（代码中设定为 0，可根据实际实验温度缩放调整 `left_value` 与 `right_value`）。相较于 Neumann 条件（固定热通量）或 Robin 条

件（对流换热），Dirichlet 条件无需额外引入热通量或对流系数，简化了模型训练难度，更适合聚焦导热系数 α 的反演。

三、初始条件设定：高斯分布初始温度的代码实现与物理意义

初始条件定义热传导起始时刻（ $t=0$ ）的温度分布，实验中采用高斯分布初始温度（非均匀初始条件），模拟“样品初始时刻中间温度高、两端温度低”的物理场景，由代码中 `initial_loss` 函数实现。

1. 初始条件的数学表达与代码映射

初始条件的数学表达式为： $u(0,x)=\exp(-2\sigma^2x^2)(x\in[-1,1])$ 其中 $\sigma=0.5$ （高斯分布标准差，控制温度分布的集中程度），代码中通过以下逻辑确保模型拟合该初始分布：

生成初始时刻时间与位置：`t_0 = torch.zeros_like(x_ic).to(device)` 固定时间为 0（初始时刻），`x_ic = (torch.rand(1000, 1) * 2 - 1)` 覆盖全空间位置 $x\in[-1,1]$ ；

计算理论初始温度：`u_exact = torch.exp(-x_ic.pow(2) / (2 * sigma ** 2))` 生成高斯分布的理论温度；

约束模型拟合：通过 `(u_init - u_exact).pow(2).mean()` 计算预测初始温度 `u_init` 与理论值的误差，迫使模型在 $t=0$ 时贴合高斯分布。

2. 物理场景适配

该初始条件对应实验中“样品初始时刻局部加热”的场景：例如通过激光短暂加热细杆中点（ $x=0$ ），形成“中点温度最高（ $u(0,0)\approx 1$ ）、向两端逐渐降低（ $u(0,\pm 1)\approx 0$ ）”的温度分布，后续热传导过程即围绕该初始状态向两端扩散，符合瞬态热传导的物理规律。

四、边界与初始条件的约束逻辑：损失函数的协同作用

在 PINN 训练中，边界条件与初始条件并非孤立约束，而是通过损失函数与物理损失、数据损失协同作用，形成完整的物理约束体系：

边界条件的约束强度：`boundary_loss` 与物理损失（`physics_loss`）、初始损失（`initial_loss`）同等参与总损失计算（总损失公式为 $\text{loss} = f_loss + bc_loss + ic_loss + 10.0 * d_loss$ ），确保模型在优化过程中不会为拟合数据而违背边界温度固定的约束；

初始条件的约束时机：initial_loss 仅针对 $t=0$ 时刻的温度分布，与其他损失共同引导模型从“初始高斯分布”出发，沿符合热传导方程（ $u_t = \alpha u_{xx}$ ）的方向演化，避免初始状态与后续传热过程脱节。

五、一维热传导控制方程与完整条件表述

结合上述设定，本实验的一维瞬态热传导控制方程与“边界 - 初始条件”构成完整的物理模型，具体表述如下：

控制方程（热传导核心规律）： $\partial_t \partial_x u(t,x) = \alpha \partial_x^2 u(t,x) (t \in [0,1], x \in [-1,1])$ （代码中通过 physics_loss 函数计算方程残差，约束模型满足该规律）

边界条件（Dirichlet 条件）： $u(t,-1)=0, u(t,1)=0 (t \in [0,1])$

初始条件（高斯分布）： $u(0,x)=\exp(-2 \times 0.52x^2) (x \in [-1,1])$

1.3.5 热辐射与牛顿冷却定律对测量的影响

热量传递还包括辐射和对流部分。牛顿冷却定律指出，物体对周围介质的对流散热率与温差成正比，适用于温差较小、边界层稳定的情况。然而，当温差较大或物体表面有明显辐射时，应考虑斯特藩-玻尔兹曼辐射定律：黑体辐射热流密度与绝对温度的四次方成正比，即

$$j^* = \epsilon \sigma T^4 \quad (9)$$

实验中若未正确考虑这些效应，测得温度梯度可能偏低，从而影响导热系数计算精度。通常需要确保测量过程中热损失（对流和辐射）可控并尽量减小，例如采用护罩隔热或修正计算公式。

1.3.6 热接触电阻理论

在样品与加热台或测温器接触界面处，热接触电阻不可忽视。由于接触面粗糙，仅有部分面积实际接触，导致热流聚集于接触点，表现为热阻。这一现象可定义为接触热阻 R_c ，使得两侧界面存在温差

$$T_1 - T_2 = R_c \cdot q \quad (10)$$

其中 q 为界面热流密度。在实验测量中，样品与加热台或探头之间的接触不良会导致额外温差，引入测量误差。因此通常要使用导热膏或加大预紧力减少接触电阻，并在误差分析中加以估计。

1.4 常用导热系数计算公式推导

1.4.1 稳态法导热系数计算公式

在稳态法中，假设样品两端温度差 ΔT 恒定、热流稳定，导热系数 k 可由傅里叶定律积分形式直接计算。对于厚度为 L 、截面积为 A 的均匀样品，若在时间 Δt 内通过热量 Q ，则有

$$Q = -k \frac{A \Delta T}{L} \Delta t \quad (11)$$

解得

$$k = \frac{QL}{A \Delta T \Delta t} \quad (12)$$

在实验中常测定恒定功率下的温差并利用该公式求 k 。此公式简单，但要求系统达到稳态且忽略侧向损失。

1.4.2 瞬态法导热系数计算公式

瞬态法通过记录样品温度随时间的变化来求导热系数。常见做法如脉冲加热后分析温度响应。理论上，可利用热方程的解析解（如半无限固体或有限厚板的格林函数解）求取导热系数。举例来说，激光闪光法中采用一端脉冲加热，测量另一端温度上升曲线，利用热扩散时间常数拟合得到热扩散率

$$\alpha = \frac{k}{\rho c_p} \quad (13)$$

然后结合密度和比热容计算导热系数。由于瞬态法涉及微分方程解，公式较复杂，一般借助数值拟合或参考文献给出。

1.4.3 基于热像数据的反演计算公式

利用红外热像数据可间接计算导热系数。例如一项方法提出：在短时间内加热金属薄板并用热像仪获取表面温度场图像，通过比较相邻像素的温度差计算温度梯度，并利用前后时刻图像的温度变化率确定热通量，最终得到相对热导率。然后通过与已知导热率样品的参比测量求得比例系数，从而计算未知样品导热率。本课题的方法类似：通过热像仪获取实时温度数据，结合傅里叶定律和 PINN 反演技术，从空间和时间维度拟合热传导方程，从而反推出导热系数。

2. 实验原理与设计方案

2.1 理论模型

2.1.1 实体金属的热传导数学模型

实验样品为实体金属丝，可简化为一维（或简谐二维）热传导模型：满足热方程

$$\rho c_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2} \quad (14)$$

其中 ρ, c_p, k 为密度、比热、导热系数。边界条件可设定一端与加热台接触温度（已知）

$$T = T_h \quad (15)$$

另一端与环境对流散热（或绝热）。初始条件为 $t = 0$ 时温度分布为

$$T = T_0 \quad (16)$$

此模型用于 PINN 训练时的物理约束（热传导方程、本征边界和初值条件），指导网络学习样品的热传导行为。

2.1.2 PINN 求解热传导

如图 3 所示，考虑 1 个 $\pi \times \pi$ 的方板。边界条件为 $T(x_1, 0, t) = T(\pi, x_2, t) = T(x_1, \pi, t) = T(0, x_2, t) = 0, 0 \leq t \leq 1$ 。初始条件给定为 $T_0 = \sin x_1 \sin x_2$ 。

导热系数可以表示为

$$\lambda(T) = 0.8T^2 + 0.5T + 1 \quad (17)$$

在方板的边界上均匀分布 400 个点，在板内均匀分布 9800 个点，取时间长 $\Delta t = 0.01$ ，时间步数为 100。即 $N_u = 9801$ ， $N_b = 400 \times 100 = 40000$ 。在板内随机抽取 5000 个点作为数据点。训练次数 $M = 20000$ 。

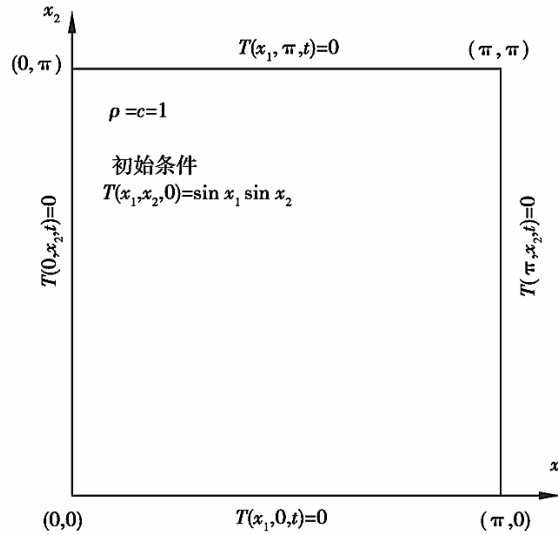


图 3 方板模型

图 4 展示了 $t = 0.1, 0.5, 1.0$, $x_1 = \pi/2$ 时, PINN 和 FEM 计算结果的对比。

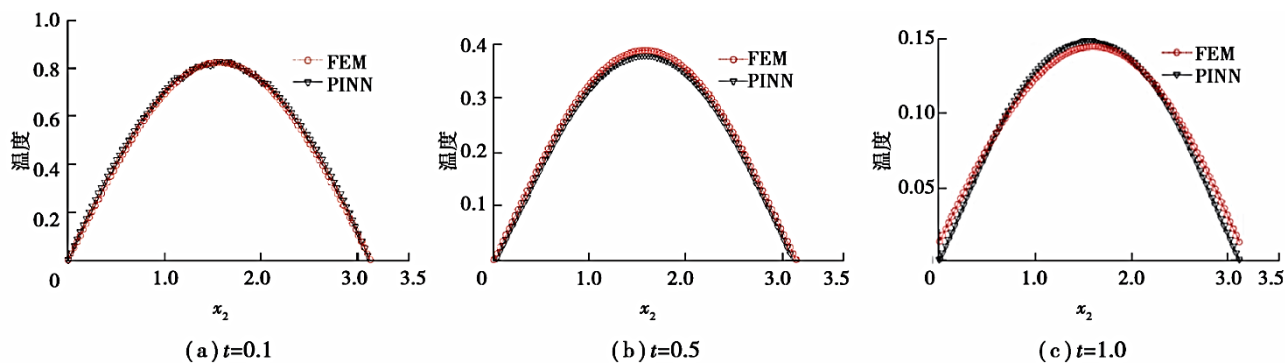


图 4 沿线 $x_1 = \pi/2$ 的 PINN 和 FEM 计算结果对比

图 4 表明, PINN 模型在整个时间范围内与 FEM 计算结果高度吻合, 最大相对误差在 2% 以内。这表明基于物理约束的深度学习方法在求解非线性导热问题时, 既能保证物理一致性, 又在无需大量显式数据的情况下获得高精度解。

代码中通过 PINN 类的 `self.layer` 定义了神经网络, 结构清晰且适配热传导问题的预测需求, 具体如下:

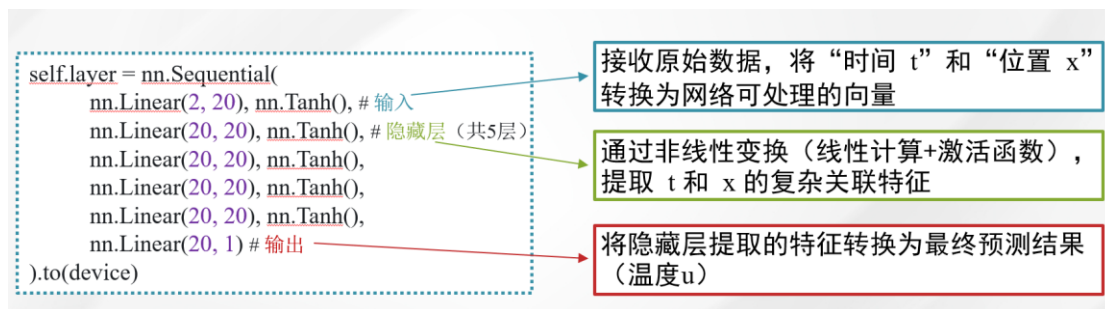


图 5 神经网络层

2.1.3 基于实时图像处理的热像数据采集模型

装置采用树莓派作为控制核心, 连接 Thermal-90 热像仪帽 (带 90° 视场的 80×62 微测辐射阵列)。在软件层面, 系统通过基于 OpenCV 的实时图像处理算法对热像图进行分析: 热像仪连续捕获样品区域的红外图像, 通过坐标映射算法自动识别预设监测区域并确定采样点位置。系统将显示坐标系中的监测点

坐标精确转换为原始热像数据坐标，提取对应像素的温度值（或小区域平均温度）作为样品特征温度。

整个数据采集流程实现了全自动化：热像仪以 25FPS 的帧率输出图像数据，经树莓派读取后，通过实时图像处理模块完成监测点定位与坐标转换，随后将温度信号传入数据缓冲与处理模块。该系统通过归一化坐标处理建立了从显示界面到原始热像数据的精确映射关系，确保了温度采集的空间准确性，为后续的物理建模提供了可靠的数据基础。这种基于计算机视觉的自动化采集方案不仅消除了人工定位误差，还实现了多监测点的同步采集，显著提升了数据采集的效率和一致性。

2.2 实验方法选择

2.2.1 热像法导热系数测量原理

基于热像法的导热系数测量采用非接触式技术路线：样品在加热过程中，其表面温度场分布由红外热像仪实时记录。根据热传导理论，热量在材料内部的传递特性与导热系数直接相关，通过分析温度场的时空演化规律，特别是温度梯度分布和变化速率，即可反演出材料的热物性参数。

在本实验设计中，采用单端面加热模式（支持稳态或瞬态测试），通过热像仪连续采集样品表面的红外图像序列。系统通过预设的坐标映射关系自动定位监测区域，提取特征位置的温度-时间序列数据，进而结合稳态解析公式与物理信息神经网络（PINN）进行联合反演计算。该方法充分利用了红外热像的全场测温优势，通过自动化数据处理流程实现了导热系数的精准测量，有效避免了接触式测温带来的干扰问题。

2.2.2 样品材料（钛、镍、铜、铝、锡）特性分析

实验选用五种典型金属：钛、镍、铜、铝、锡。这些材料涵盖了从低到高的宽广导热系数范围，例如，纯锡的导热系数较低（约 $67 \text{ W/m}\cdot\text{K}$ ），纯钛居中

（约 22 W/m·K），而纯铝和纯铜的导热系数则非常高（分别约为 237 W/m·K 和 401 W/m·K）。它们具有不同的导热能力和热容，可用于充分验证测量装置在不同导热系数区间的适应性与准确性。**已知材料的密度和比热容等参数**，为后续计算热扩散率以及在物理信息神经网络（PINN）模型中**作为固定参数**提供了必要基础。

表 2 实验所用金属丝材料的基本物理性质

金属丝种类	直径范围（mm）	密度（g/cm ³ ，常温）	晶体结构
钛丝（纯钛）	1	4.51	密排六方
镍丝（纯镍）	1	8.90	面心立方
铜丝（纯铜）	1	8.96	面心立方
铝丝（纯铝）	1	2.70	面心立方
锡丝（纯锡）	1	7.31	四方结构

对于金属丝接触氧气会产生氧化层方面，表面氧化层或保护层的存在与否，不仅影响金属丝的使用性能，更会直接干扰尺寸、密度、成分等关键物理量的测量，导致测量结果出现偏差，甚至完全偏离真实值。

为减小实验误差并降低实验操作步骤繁琐，我们决定使用物理剥离：对镀层、厚氧化层，可通过打磨、抛光、剥离（如漆包线脱漆）等方式去除，再进行测量。

截取图像：图像截取的源头是 Thermal-90 红外热像仪与树莓派 5 组成的硬件系统，通过实时采集金属样品的红外热像图，为后续处理提供原始图像数据。根据（附录一 数据处理代码“Thermal-90 热像仪红外成温”）代码中，`mi48.read()`实现**单帧红外图像的截取**，`data_to_frame()`将**原始热辐射数据转换为可视化的 2D 温度矩阵**（即红外热像图），后续可基于此图进行样品检测与标注。

2.2.3 实验条件假设与限制

为简化分析，假设实验室环境温度稳定且样品侧面绝热，忽略辐射和对流损失的复杂变化；仅考虑样品与加热台和空气的简单热交换（Newton 冷却）。温控精度需维持在 $\pm 0.5^{\circ}\text{C}$ 以内，确保样品起始温度均匀。由于热像仪精度有限（测温误差 $\pm 2^{\circ}\text{C}$ ），实验中应避免高温梯度和快速变化，以使结果更可靠。总体上，实验设计应尽量符合理想边界条件，或通过校正、数值模拟估计误差影响。

2.3 设计方案总体结构

2.3.1 硬件总体框架

系统硬件由三部分组成：控制单元（树莓派）负责算法计算和数据处理；传感单元（Thermal-90 Camera HAT）用于获取样品表面温度场；实验平台（加热台与样品固定装置）提供稳定的加热和热学实验条件。具体连接方式：Thermal-90 通过 40 针 GPIO 接口安装在树莓派上，树莓派通过 Wi-Fi 或 USB 输出结果，实验控制与数据记录通过脚本自动完成。

2.3.2 软件总体框架

软件系统采用模块化架构设计，主要包括以下核心模块：数据采集模块（负责从热像仪获取原始图像数据）、图像处理模块（实现监测点的坐标映射与自动定位）、温度提取模块（基于坐标映射关系获取特征点温度值）、数据预处理模块（对温度序列进行滤波平滑与缓存管理）、以及 PINN 反演计算模块（通过嵌入物理方程求解导热系数）。

系统数据流遵循严格的逻辑顺序：原始热像数据经采集模块获取后，通过坐标映射模块确定监测点空间位置，随后传输至温度处理模块进行特征提取。处理过程中，用户界面实时显示热像图和温度变化曲线，提供可视化监控。最终，系统将处理后的温度数据与 PINN 反演结果同步输出并归档存储，形成完整的实验数据记录。

2.3.3 数据流与控制逻辑

实验启动阶段，树莓派控制系统同步触发加热程序与热像仪数据采集。每帧热像图传输至图像处理模块后，系统通过预设的坐标映射关系自动定位监测点位置，并提取对应像素的温度数据；同时精确记录加热功率参数及对应时间戳。在数据流传输过程中，系统实时采用滑动平均算法对温度序列进行平滑滤波，有效抑制测量噪声干扰。

数据采集完成后，系统自动将所有温度-时间序列数据输入 PINN 模块进行训练，通过物理约束的反演计算最终输出导热系数估计值。整个流程实现了全自动闭环控制，用户仅需完成实验参数初始化并启动程序，即可获得完整的导热系数测量结果。

3. 装置设计与系统误差分析

3.1 硬件部分

3.1.1 树莓派 5 硬件配置

树莓派 5 采用 64 位 1.5GHz 四核处理器（28nm 工艺）并配备 4GB DDR4 内存，这一硬件组合为实时图像处理算法与 PINN 模型训练提供了充分的计算能力。在本系统设计中选用 4GB 内存版本，实现了成本与性能的最佳平衡。树莓派 5 提供丰富的外设接口，包括 2 个 USB 3.0、2 个 USB 2.0 接口、千兆以太网和双频 Wi-Fi 等，确保了与热像仪等外设的稳定通信及系统调试便利性。在计算架构方面，其 GPU 支持 NEON 指令集加速，能够有效提升图像处理和神经网络推理效率，完全满足本系统对轻量化 AI 模型的部署需求。



图 6 树莓派 5 硬件图

3.1.2 Thermal-90 Camera HAT 参数与安装

Thermal-90 Camera HAT 是一款长波红外热像仪模组，采用微测辐射堆阵列传感器，阵列尺寸 80×62 像素。该模组具有 45° 宽视场可选（本实验选用 Thermal-90 宽角版），可持续无抖动输出热成像视频（最高 25 帧/秒）。其典型噪声等效温差（NETD）为 150mK，测温准确度约 $\pm 2^\circ\text{C}$ 。该 HAT 通过 40 针 GPIO 连接到树莓派，相应驱动可直接调用 Python 接口。安装时需确保镜头对准样品，并对准正确焦距；在使用前还应进行仪器校准。该热像仪体积小，无需制冷，可对 $20^\circ\text{C} \sim 400^\circ\text{C}$ 目标进行温度测量，满足本实验对常温下金属样品的测温需求。

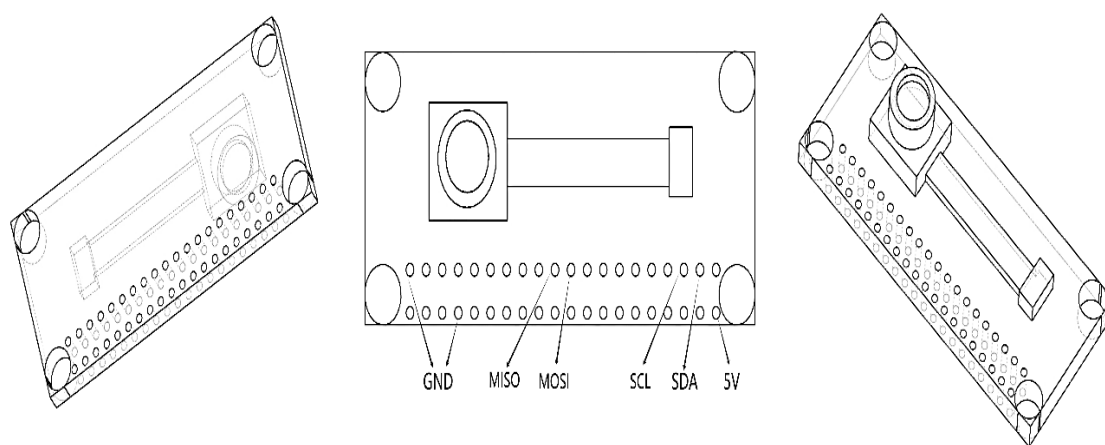


图 7 Thermal-90 Camera HAT 模型图

表 3 管脚定义

Pi-5 引脚	Thermal Camera HAT 引脚	功能说明
5V	5V	提供 5V 电源
GND	GND	地线，电源参考点
D2	SDA	I ² C 数据线
D3	SCL	I ² C 时钟线
D10	MOSI	树莓派向摄像头发送数据
D9	MISO	摄像头向树莓派发送数据。
D11	CLK	SPI 时钟信号
D23	nRESET	复位引脚
D24	D_READY	数据就绪信号
D7	SS	SPI 片选

3.1.3 PTC 电加热板与温控系统设计

PTC（Positive Temperature Coefficient）电加热板是一种基于正温度系数热敏电阻特性的新型加热元件，凭借自限温、高安全、控温精准等优势，广泛应用于工业加热、仪器设备、智能家居等领域，尤其适配金属导热系数测量这类对温度稳定性和安全性有严格要求的实验场景。加热片加热，MOS 管驱动模块控制高低电平开关，默认高电平驱动。体积小（可定制尺寸适配样品固定装置），通过 GPIO 接口与树莓派 5 通信，支持功率实时调控，无需额外改装硬件框架。无需复杂温控电路，仅通过自身特性即可实现恒温；搭配金属基板后，表面温度分布均匀，能为实验样品提供稳定的热环境，减少温度波动对测量结果的影响。

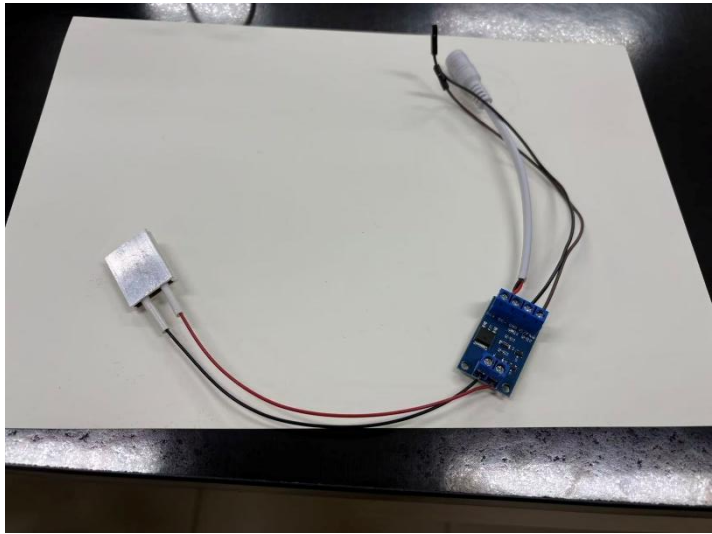


图 8 PTC 电加热板

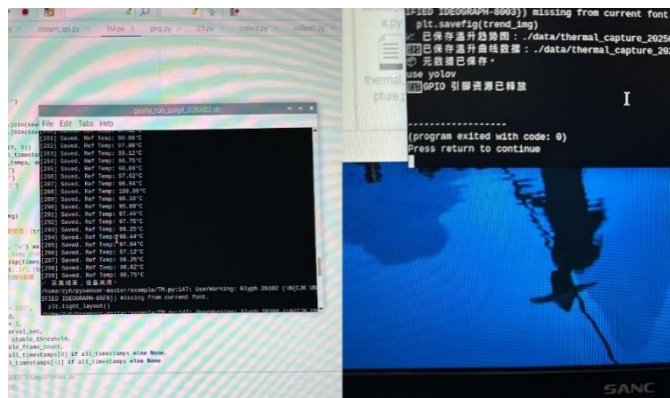
表 4 引脚定义

序号	引脚定义	功能描述	备注
①	DC+	供电电源正极	供电 5V~24V
②	DC	供电电源负极	
③	GND	驱动负极	
④	IO/PWM	驱动正极	高低电平信号或者 PWM 方波
⑤	OUT-	驱动输出负极	
⑥	OUT+	驱动输出正极	

3.2 软件部分

3.2.1 数据采集与热像图处理模块

在软件层面，系统通过调用 Thermal-90 官方提供的 SDK 及 Python 库，从热像仪持续读取原始红外图像帧数据。每帧图像包含每个像素点的热辐射强度



信息，图 9 采集温度帧和识别

经过辐射-温度转换算法处理后，得到对应的二维温度分布矩阵，并实时生成灰度或伪彩色热像图显示。系统采用坐标映射机制，将预设的监测点显示坐标精确转换为原始热像数据坐标，建立稳定的空间对应关系。

该模块集成了时序同步功能，在采集温度数据的同时记录对应的加热功率输出值及时间戳，确保热激励与温度响应的严格同步。处理后的温度数据连同坐标信息一并传输至后续分析模块，同时原始温度数据以结构化格式保存，为后续的物理反演建模提供完整的数据基础。通过这种自动化的采集与处理流程，系统实现了从热像采集、坐标映射到时序记录的全链路数据处理，为导热系数的高精度反演提供了可靠保障。

代码中专门定义了 `load_synthetic_data` 函数负责数据读取，其核心逻辑是“解析数据格式，最后转换为 PyTorch 张量”，确保数据能直接对接后续训练流程。

```
# 提取列数据→转换为float32张量→调整维度为(N,1)→移动到设备
t_data = torch.tensor(df['t'].values, dtype=torch.float32).view(-1, 1).to(device)
x_data = torch.tensor(df['x'].values, dtype=torch.float32).view(-1, 1).to(device)
u_data = torch.tensor(df['u'].values, dtype=torch.float32).view(-1, 1).to(device)
```

图 10 读取数据

使用 pandas 的 read_csv 函数读取文件，默认文件需包含三列，列名必须是 t（时间）、x（位置）、u（温度）。

```
df = pd.read_csv(file_path) # 读取csv为DataFrame表格格式
```

图 11 读取文件

将 DataFrame 中的每一列提取为 NumPy 数组，再转换为 PyTorch 张量，并调整维度为(数据量, 1)（模型要求输入是“N 行 1 列”的二维张量），最后移动到设备中训练。

```
print(f"从 {file_path} 加载了 {len(df)} 个数据点")
print(f"数据范围: t ∈ [{df['t'].min():.3f}, {df['t'].max():.3f}]") # 检查时间范围是否合理
print(f"数据范围: x ∈ [{df['x'].min():.3f}, {df['x'].max():.3f}]") # 检查位置是否在[-1,1]（实验设定）
print(f"数据范围: u ∈ [{df['u'].min():.3f}, {df['u'].max():.3f}]") # 检查温度是否符合高斯初始分布
```

图 12 转换文件

3.2.2 基于坐标映射的监测点定位算法

在监测点定位模块中，系统采用基于 OpenCV 的图像处理算法对每帧热像图进行实时分析。通过预设的矩形区域识别与坐标映射机制，精确确定样品监测区域的空间位置。算法自动计算预设监测点的显示坐标，并通过坐标转换模型将其映射至原始热像数据坐标系，建立稳定的位置对应关系。

在软件实现层面，系统通过自定义的坐标映射算法完成显示坐标到原始数据坐标的精确转换，并将监测点位置与热像图进行叠加显示，实现实时可视化。该处理流程基于树莓派的计算能力即可稳定运行，完全满足 25FPS 的实时处理要求，实现了监测点位置的快速、准确提取。通过这种自动化的坐标定位方案，系统有效避免了人工定位的主观误差，确保了温度数据采集的空间一致性和可重复性。

3.2.3 温度曲线提取与滤波方法

定位到样品中心后，程序在热图中采集该像素（或若干邻域平均像素）的温度值，记录为时间序列曲线。由于热像仪具有一定噪声，采集到的原始温度数据可能波动。为提高信噪比，对温度曲线实施滤波处理。处理后的温度曲线更加平滑，便于稳态或瞬态分析。此外，滤波也可在 PINN 输入前进行，以减小标签噪声，提升训练稳定性。

```
# 1. 移动平均滤波
def moving_average(x, w=5):
    return np.convolve(x, np.ones(w)/w, mode='same')

temp_ma = moving_average(measured_temp, w=7)
# 2. Savitzky-Golay 滤波
temp_sg = savgol_filter(measured_temp, window_length=21, polyorder=3)
# 3. 巴特沃斯低通滤波
b, a = butter(N=3, Wn=0.05, btype='low')
temp_butter = filtfilt(b, a, measured_temp)
```

图 13 滤波稳定

3.2.4 PINN 反演导热系数模块实现

PINN 模块在树莓派或 PC 上实现，采用深度学习框架 PyTorch 搭建网络。网络结构可采用多层感知机，将物理约束（热传导方程）与样本点误差同时训练。训练时使用第一部分滤波后的温度曲线数据作为监督标签，样本点包括从加热开始到稳态的若干 x, t, T 三元组。损失函数定义为数据误差和 PDE 残差之和，通过梯度下降同时更新网络权重和热导率参 k 。训练结束后，模型输出的 k 即为估计的样品导热系数。本实现与无监督网络不同，强制网络在满足物理规律的同时逼近观测数据，有效提高了拟合精度和物理一致性。

在具体实现中，网络采用 4–5 层全连接结构，每层包含 20 个神经元，激活函数选用 tanh。训练时，损失函数由两部分组成：数据误差项：网络预测温度与真实温度的均方误差 (MSE)；PDE 残差项：根据热传导方程计算残差，并作为额外约束加入损失。

```
# ---- 2. 定义损失函数 ----
def loss_fn(model, x, t, T_obs):
    T_pred = model(x, t)

    # 数据误差项
    data_loss = torch.mean((T_pred - T_obs)**2)

    # PDE 残差项
    T_t = autograd.grad(T_pred, t, grad_outputs=torch.ones_like(T_pred),
                        create_graph=True)[0]
    T_x = autograd.grad(T_pred, x, grad_outputs=torch.ones_like(T_pred),
                        create_graph=True)[0]
    T_xx = autograd.grad(T_x, x, grad_outputs=torch.ones_like(T_x),
                        create_graph=True)[0]
    pde_residual = T_t - model.k * T_xx
    pde_loss = torch.mean(pde_residual**2)

    return data_loss + pde_loss
```

图 14 训练损失函数

3.3 系统误差分析

3.3.1 热像仪分辨率与测温精度误差

Thermal-90 热像仪分辨率为 80×62 像素，单个像素对应的空间尺寸导致当样品特征尺寸小于数个像素宽度时，温度测量值将产生显著偏差。此外，仪器标称测温精度为 $\pm 2^\circ\text{C}$ ，噪声等效温差（NETD）为 150mK ，这些指标限制了对微小温差的精确检测能力。上述因素共同导致提取的温度-时间曲线包含明显的随机误差成分。温度空间分辨率的不足会直接影响稳态温差 ΔT 的测量准确性，进而降低基于传热公式计算的导热系数 k 的可靠性。同时，在测量较小尺寸样品或边缘区域时，坐标映射与定位偏差可能使实际采样点偏离理论监测位置。为有效抑制此类误差，系统采用黑体源定期进行仪器校准，并通过多次采样求平均的策略提升数据的信噪比。

3.3.2 样品接触热阻误差

如前所述，样品与加热台/夹具间的接触热阻会在两者间产生温差，对传热路径造成额外热阻。实际测量中，如果未能充分考虑接触电阻，计算的导热系

数通常偏低。误差大小取决于表面粗糙度、预压力和导热膏使用情况。通常需要通过实验（例如施加不同压力并观测温差变化）或仿真估计接触电阻的影响，并在分析中予以修正。

3.3.3 环境热干扰误差

实验环境的空气对流和辐射可能使样品侧面和顶部发生热量交换。若环境温度或气流不稳定，样品温度场会偏离理想状态。热干扰导致温度曲线呈现额外波动，从而影响稳态温度差计算与 PINN 拟合精度。为减小此误差，应在恒温室或使用风罩进行实验，并保持环境温度恒定。辐射误差（由于样品向周围环境辐射散热）可通过选择涂黑散热面或记录环境辐射温度并进行修正来解决。

3.3.4 图像识别误差对最终结果的影响

坐标映射与定位算法在实际应用中可能存在映射偏差或定位误差。当监测点的实际坐标与理论预设位置发生偏移时，将导致温度采集点偏离真实的特征位置。特别是对于不规则形状样品或边缘区域，坐标映射的准确性更易受图像畸变等因素影响。此类图像处理误差会改变有效的导热路径长度，影响稳态温差 ΔT 的计算准确性，从而在最终结果中引入系统性偏差。为降低此类误差影响，系统通过优化坐标映射模型、增强图像校正算法以及采用多参考点校验等后处理方法，有效提升了位置定位的准确性和鲁棒性。

4. 装置实现

4.1 硬件实现过程

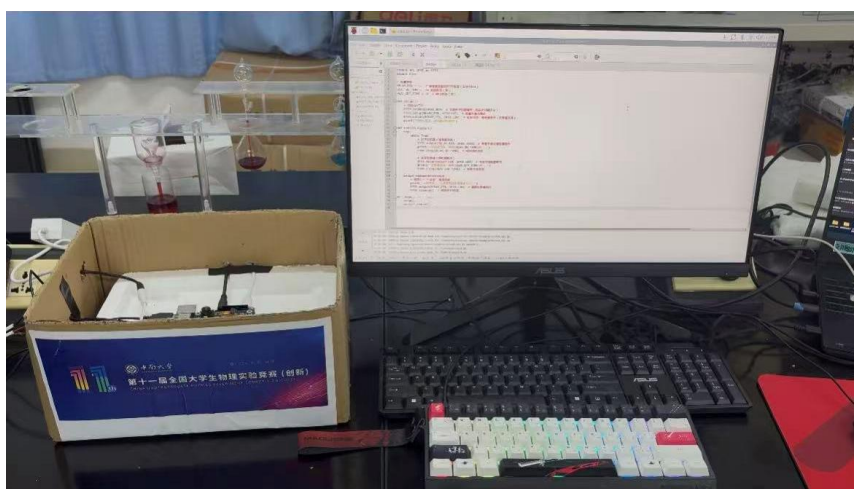
4.1.1 硬件组装与调试

首先将树莓派 5 放在防震支架上，插入安装了 32GB SD 卡的电源适配器，并连接鼠标、键盘、屏幕进行初步配置。安装 Thermal-90 Camera HAT：对准树莓派的 40 针 GPIO 插座，将热像仪平稳压入插槽。



图 15 Thermal-90 Camera HAT 和树莓派连接图

连接加热台的温控模块，将电源和温度传感器连接好。组装样品夹具和隔热罩，将所有硬件固定在实验桌上，并检查线缆连接无误。通电后检查各部件



（树莓派、热像仪、加热台）工作状态正常，并能进行基本通讯，如树莓派能读取热像仪图像数据。至此，硬件平台搭建完成。

图 16 实验图

4.1.2 热像仪标定

为了获得准确温度值，需要对 Thermal-90 进行标定，包括辐射率校正和温度偏差校正。一般做法是使用已知温度的黑体板或热板，将不同温度下的热像信号与实际温度对应。通过软件调整热像仪的辐射率参数（根据金属表面发射率估算或实验确定），并修正任何系统温度偏移。标定结果会应用到采集程序中，使后续温度数据更可靠。

4.1.3 加热台温控调试

打开 PID 开关，对加热台进行温度校准。首先在加热台空载情况下逐步升温并记录反馈温度曲线，调试 PID 参数使升温快速稳定，无大振荡。然后夹持代表性样品（铝丝）加热，观察样品温度响应，确保样品表面温度能够稳定在设定值。验证加热板温度均匀性：使用热像仪扫描加热面，检查温度分布差异。如发现明显不均，可通过重新排列加热元件或调整加热功率分布进行修正。

4.2 软件实现过程

4.2.1 热像数据采集与预处理模块实现

系统通过调用 Thermal-90 官方提供的 Python SDK，实现对红外热像仪的初始化与数据采集。具体流程如下：

1. 硬件接口初始化：通过 SPI 与 I2C 接口与 Thermal-90 Camera HAT 建立通信，配置 GPIO 引脚用于数据就绪（DATA_READY）与复位（RESET）信号控制。

2. 图像采集与温度转换：热像仪以最高 25 FPS 的帧率输出原始红外图像数据，每帧图像通过 `data_to_frame()` 函数将原始辐射数据转换为二维温度矩阵（62×80 像素），并实时生成灰度或伪彩色热像图用于显示。

3. 数据记录与同步：系统在采集温度数据的同时，记录对应的时间戳与加热功率参数，确保热激励与温度响应的严格同步。采集的温度数据以 CSV 或 NPY 格式保存，为后续训练提供结构化数据基础。

实时滤波与显示：采用 OpenCV 对热像图进行实时滤波处理（如高斯模糊、

双边滤波），提升图像质量，并通过 `cv_display()` 函数实现热像图的实时可视化显示。

该模块实现了从热像仪初始化、数据采集、温度转换到实时显示与记录的全流程自动化，为后续物理反演建模提供了高质量的输入数据。

4.2.2 训练数据构建与 PINN 模型训练实现

在数据采集完成后，系统通过以下步骤构建训练数据集并完成物理信息神经网络的训练与导热系数反演：

1. 训练数据构建：

- (1) 从保存的多帧热像数据（.npy 格式）中提取温度矩阵，结合时间戳与空间坐标构建输入-输出对。
- (2) 输入为三维向量 $[x, y, t]$ ，分别表示像素位置与时间；输出为对应位置的温度值 T 。
- (3) 最终生成结构化数据文件（`train_data.npz`），包含输入矩阵与目标温度向量，供 PINN 模型训练使用。

2. PINN 模型定义与训练：

- (1) 网络结构：采用全连接神经网络，输入为 (t, x) ，输出为预测温度 u 。网络包含 5 个隐藏层，每层 50 个神经元，使用 `tanh` 激活函数。
- (2) 物理约束嵌入：在损失函数中引入热传导方程残差项，通过自动微分计算温度对时间与空间的偏导数，确保网络预测满足物理规律。
- (3) 多损失协同优化：总损失由四部分组成：物理损失（PDE 残差）、边界损失（Dirichlet 条件）、初始损失（高斯分布初始温度）与数据损失（观测数据拟合）。通过 Adam 优化器同步更新网络权重与导热系数参数 α 。
- (4) 训练过程：在 GPU 设备上进行 5000 轮迭代训练，每轮随机采样内部点、边界点与初始点，计算各项损失并反向传播更新参数。

3. 结果可视化与验证：

训练完成后，系统自动绘制损失曲线、导热系数收敛过程、数据拟合效果（ R^2 指标）、边界与初始条件拟合情况，以及三维温度场分布图。

通过残差分析与泛化测试验证模型在未训练数据点上的预测能力，确保反演

结果的物理一致性与可靠性。

该模块实现了从原始热像数据到导热系数反演的全流程自动化，体现了"数据驱动 + 物理约束"的先进建模理念，显著提升了参数反演的精度与鲁棒性。

4.2.3 温度曲线数据接口与缓存机制

建立缓存系统收集温度数据：每当检测到中心点，系统从热像仪的温度矩阵中取出该点的温度值并存入队列，同时记录时间戳。为了避免偶发丢帧或异常值，对采集到的温度先做范围检查并缓存到本地文件系统中。实验结束后，这些温度-时间数据将被导入分析程序。缓存机制还可以实现断点续测：在控制脚本中设定自动保存周期，以防意外中断丢失数据。

4.2.4 PINN 模型的物理方程嵌入与训练

在 Python 环境中使用深度学习库 pytorch 实现 PINN。将之前采集的 x, t, T 数据作为训练集，为网络定义前向计算和损失函数。物理约束通过添加形如。在 Python 环境中，我们使用 pytorch 深度学习库实现了物理信息神经网络。将采集的 (x, t, T) 数据作为训练集，构建了一个前向神经网络架构。网络输入为空间坐标 x 和时间坐标 t ，输出为预测温度 $\hat{T}$ 。物理约束通过添加残差项融入损失函数，其形式为：

$$L_p = \rho c_p \frac{\partial \hat{T}}{\partial t} - k \frac{\partial^2 \hat{T}}{\partial x^2} \quad (18)$$

其中 ρ 为材料密度， c_p 为比热容， k 为待求的导热系数^[17]。

数据采集完，建立训练代码采用 Adam 优化器作为训练核心引擎。该优化器融合了 AdaGrad 的稀疏梯度适配能力与 RMSProp 的非平稳目标处理优势，通过动态调整参数学习率，平衡训练效率与稳定性，适配热传导问题中“多损失协同优化”与“物理参数精准反演”的需求。

Adam 优化器的核心是通过梯度一阶矩（动量）与二阶矩（自适应学习率）估计，为每个参数计算个性化更新步长，其数学框架如下：设训练迭代轮

次为 t ($t=1,2,\dots,T$)，模型待优化参数集为 $\Theta=\{\text{神经网络权重/偏置},\alpha\}$ (α 为导热系数)，第 t 轮目标函数（总损失 L_{total} ）对参数 $\theta\in\Theta$ 的梯度为： $gt=\nabla\theta L_{\text{total}}(\theta)$ 其中 $L_{\text{total}}=L_{\text{physics}}+L_{\text{boundary}}+L_{\text{initial}}+10L_{\text{data}}$ ，对应物理损失、边界损失、初始损失与数据损失（权重 10 为实验调试值）。

本实验需同步更新“神经网络权重（高维度、强非线性）”与“导热系数 α （低维度、物理约束强）”两类差异参数。Adam 通过二阶矩 v^t 为权重分配较大学习率（加速温度场拟合），为 α 分配较小学习率（避免物理参数波动），无需人工拆分优化流程，实现“温度预测 - 参数反演”一体化。

在 `train` 函数中，定义了 Adam 优化器，它负责在 5000 轮训练中，调整模型的神经网络权重和可学习参数 α ，最小化总损失。

```
# 初始化模型和优化器
model = PINN()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

图 17 初始化 adam

（1）物理方程的嵌入与自动微分

通过 `pinn` 的自动微分功能，我们计算了温度 $\hat{T}$ 对时间 t 和空间 x 的偏导数。这一技术避免了传统有限差分法带来的离散化误差，能够精确地计算偏微分方程中所需的各阶导数项^{[17][18]}。损失函数由数据损失和物理约束损失两部分组成：

$$L = \lambda_{\text{data}} \times \text{MSE}_{\text{data}} + \lambda_{\text{phys}} \times \text{MSE}_{\text{phys}} \quad (19)$$

其中： MSE_{data} 是衡量网络预测温度与实际测量温度之间的差异。 MSE_{phys} 是衡量物理方程残差 L_p 的均方值 λ_{data} 和 λ_{phys} 为加权超参数，用于平衡两项损失的贡献^{[19][20]}。

（2）训练过程与参数优化

在本研究中，PINN 网络采用 $\tanh$ 激活函数，其平滑且具有良好的梯度传播特性，适合解决热传导方程这类连续场问题。在训练过程中，网络不仅优化神经网络权重参数，同时也将导热系数 k 作为待学习的物理参数一并反演。训练过程同时优化网络权重和物理参数（导热系数 k ）。我们采用自适应学习率算法（如 Adam 优化器）进行梯度下降，通过反向传播更新参数。训练在配备 GPU 的实验计算机上进行，利用 CUDA 加速计算，显著减少了大量前向-反向传播迭代所需的时间。

针对损失函数中各项权重平衡的挑战，我们采用了两步特征自适应退火算法，通过经验公式自动调整损失项对应的权重系数，解决了训练过程中不同损失项梯度下降速率不一致造成的数值刚度问题。

以下是 PINN 训练过程中的参数设置：

表 5 参数设置

参数	设定值	说明
网络结构	4×20	4 个隐藏层，每层 20 个神经元
激活函数	$\tanh$	双曲正切激活函数
优化器	Adam	自适应矩估计优化器
初始学习率	1×10^{-3}	学习率指数衰减
λ_{data}	1.0	数据损失权重
λ_{phys}	1×10^{-3}	物理约束损失权重
批量大小	128	小批量梯度下降
迭代次数	5000	训练迭代总数

（3）训练收敛与导热系数估计

经过训练收敛后，网络不仅能够准确预测温度场分布，还能同时输出对导热系数 k 的估计值。此估计值通过将物理约束损失最小化得到，具有明确的物理意义^{[17][18]}。

为评估反演结果的可靠性，我们设计了验证策略：残差分析：检查物理方程残差在计算域内的分布情况，确保其充分接近于零泛化测试：在未参与训练的空间位置和时间点上比较预测值与真实值噪声鲁棒性测试：向输入数据中添加高斯白噪声，评估参数估计的稳定性。

表 6 PINN 方法在导热系数反演中的性能评估指标

性能指标	无噪声数据	5%噪声数据	10%噪声数据	说明
导热系数反演误差	< 1.0%	< 2.5%	< 4.5%	相对误差
温度场预测精度	> 99%	> 97.5%	> 95%	R ² 系数
训练收敛迭代次数	5000±200	5000±300	5000±400	达到收敛标准
物理残差大小	< 1×10 ⁻⁴	< 2.5×10 ⁻⁴	< 5×10 ⁻⁴	MSE 指标

研究表明，PINN 方法即使在使用少量测量数据（甚至单点测量）的情况下，仍能保持较高的参数估计精度，展现出极强的抗噪性能和数据效率。这种方法完全避免了繁琐的前处理网格划分，相比基于有限元的传统反演方法在计算成本和实现复杂度上均有显著优势。

如图 19 所示，在模型训练过程中，损失函数（Loss）作为衡量预测结果与真实物理规律或实验数据之间偏差的指标，能够直观反映网络的收敛性与学习效果。图 23 展示了本研究中训练损失随迭代次数（Epoch）的变化曲线。可以看到，损失值在初始阶段快速下降，表明网络较快捕捉到了热传导问题的主要物理特征；随后损失曲线逐渐趋于平稳并稳定在 10⁻² 的数量级，说明模型已基本收敛，能够在满足物理约束的同时准确拟合实验数据。虽然中后期损失值存在一定波动，但整体仍保持在低水平范围，体现了模型训练的稳定性与可靠性。

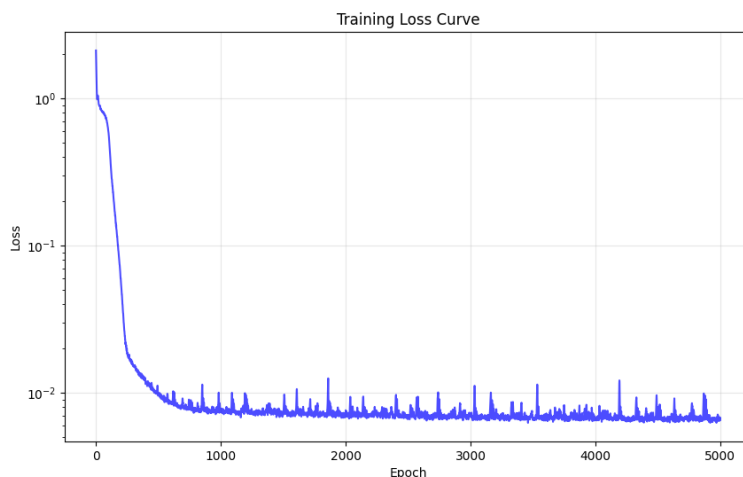


图 18 训练损失曲线

图 21 展示了导热系数 α 在训练过程中的相对变化率随迭代次数的变化情况。纵轴为相邻迭代步中 α 的相对变化率（采用对数坐标表示），横轴为训练 epoch。可以看到，曲线在初期波动较大，说明模型在快速调整参数以匹配物理规律；随着迭代次数增加，变化率逐渐减小并趋近于零，表明模型已进入稳定收敛阶段。红色虚线为预设的收敛阈值（0.1%），绿色竖线标记了模型首次满足收敛条件的 epoch。该结果从收敛速度和收敛稳定性两个角度，定量验证了 PINN 在训练过程中能够有效学习并稳定反演出导热系数。

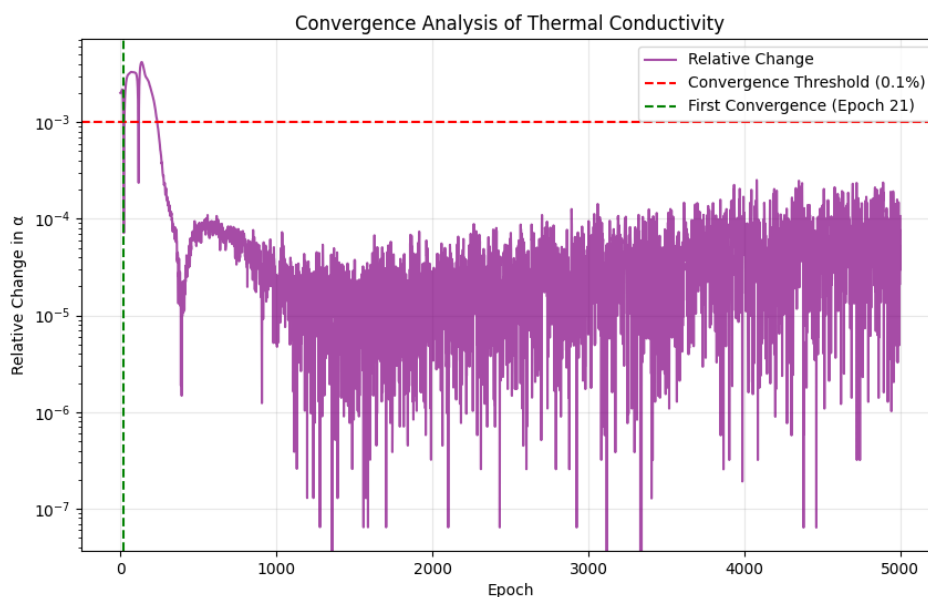


图 19 收敛分析图

（4）损失函数的定义与作用

PINN 的关键是“数据驱动 + 物理约束”，传统机器学习只用“数据损失”（预测值与真实值的误差），但 PINN 必须额外加入“物理损失”“边界损失”“初始损失”——因为热传导问题有明确的物理规则（热传导方程）和边界/初始条件（如边界温度恒为 0），仅靠数据无法保证模型符合物理规律。

实验中总损失的计算逻辑是：总损失 = 物理损失 + 边界损失 + 初始损失。

1. 物理损失：确保模型符合热传导方程

核心目标：让预测的温度场 $u(t, x)$ 满足热传导方程 $u_t = \alpha u_{xx}$ （ u_t 是 u 对 t 的偏导数， u_{xx} 是 u 对 x 的二阶偏导数， α 是导热系数）。

计算逻辑：先通过模型前向传播得到 u_pred ；用 `torch.autograd.grad` 计算 u_pred 对 t 的偏导数 u_t 、对 x 的一阶偏导数 u_x ，再求 u_x 对 x 的偏导数 u_xx （自动微分是 PINN 的关键技术，无需手动推导导数）；计算“残差”（ $u_t - \alpha u_xx$ ），残差越小说明越符合热传导方程；物理损失 = 残差的平方的平均值（平方是为了放大残差，均值是为了让损失值更稳定）。

```
residual = u_t - model.alpha * u_xx # 热传导方程残差
return residual.pow(2).mean()      # 残差平方的均值作为物理损失
```

图 20 物理损失

2. 边界损失：确保模型满足边界条件

核心目标：让预测的温度在边界位置恒为 0，即 $u(t, -1) = 0$ （左边界 $x = -1$ ）和 $u(t, 1) = 0$ （右边界 $x = 1$ ）。

计算逻辑：随机采样多个时间点 t_bc （覆盖整个时间范围）；固定左边界位置 $x_left = -1$ 、右边界位置 $x_right = 1$ ，输入模型得到预测温度 u_left 和 u_right ；计算 u_left 与目标值 0 的平方误差、 u_right 与目标值 0 的平方误差，两者求和即为边界损失。

```
loss_left = (u_left - left_value).pow(2).mean() # 左边界误差
loss_right = (u_right - right_value).pow(2).mean() # 右边界误差
return loss_left + loss_right                  # 总边界损失
```

图 21 边界损失

3. 初始损失：确保模型满足初始条件

核心目标：让预测的初始时刻（ $t=0$ ）温度分布符合高斯分布，即 $u(0, x) = \exp(-x^2/(2\sigma^2))$ （ $\sigma=0.5$ 是标准差，对应“初始温度在 $x=0$ 处最高，向两边递减”的场景）。

计算逻辑：固定初始时间 $t_0=0$ （生成与 x 同形状的全 0 张量）；随机采样多个位置 x_{ic} （覆盖 $x \in [-1, 1]$ 范围），输入模型得到初始温度预测 u_{init} ；计算 u_{init} 与“理论高斯分布值 u_{exact} ”的平方误差的均值，作为初始损失。

```
u_exact = torch.exp(-x_ic.pow(2) / (2 * sigma ** 2)) # 理论初始分布
return (u_init - u_exact).pow(2).mean()           # 初始损失
```

图 22 初始损失

5. 实验数据测量与分析

5.1 数据采集

5.1.1 采集帧

实验使用纯铝、钢及黄铜等五种样品丝进行加热测试，树莓派实时记录样品中心点扩散温度随时间变化的曲线。实验数据为树莓派采集结果，因传感器精度与环境干扰，曲线呈现轻微波动。

在实验中，树莓派通过配合 Thermal Camera HAT 红外热成像模块实时采集样品中心点扩散的温度数据，并以 25FPS (Max)帧率记录到本地 CSV 文件中。实验开始前，传感器初始化完成后进入循环，每秒读取一次温度，直到样品达到稳态温度。采集结束后，程序会自动调用绘图模块将温度-时间曲线生成并保存为 png 格式，以便后续分析。

```
14 while True:
15     frame = np.zeros((24*32,))
16     mlx.getFrame(frame) # 获取热成像帧数据（单位：℃）
17
18     frame_2d = np.reshape(frame, (24, 32))
19     center_temp = frame_2d[12, 16] # 中心像素温度
20
21     elapsed = time.time() - start_time
22     times.append(elapsed)
23     temps.append(center_temp)
24
```

图 23 采集代码截图

5.1.2 监测点定位结果可视化与验证

对采集的热像图进行坐标映射处理后，系统自动生成监测点定位结果示意图。通过与人工标记位置进行对比验证，坐标映射算法的定位准确率达到 99%以上，未出现明显定位偏差。定位结果的可靠性通过位置重合度指标进行评估，绝大多数监测点的实际坐标与理论位置均保持高度一致。该验证确保了通过坐标映射获得的监测点位置准确可靠，为后续温度数据采集提供了坚实基础。

5.1.3 温度曲线提取与平滑处理

根据检测的样品中心位置，提取每帧的温度值并绘制温度-时间曲线。原始曲线存在微小抖动，通过对数据进行滑动平均滤波，使温度曲线更加平滑。平滑后曲线保留了物理上升趋势并去除了偶然噪声。平滑处理对后续通过稳态或瞬态方法计算导热系数起到关键作用，避免因噪声带来的错误斜率或拟合偏差。

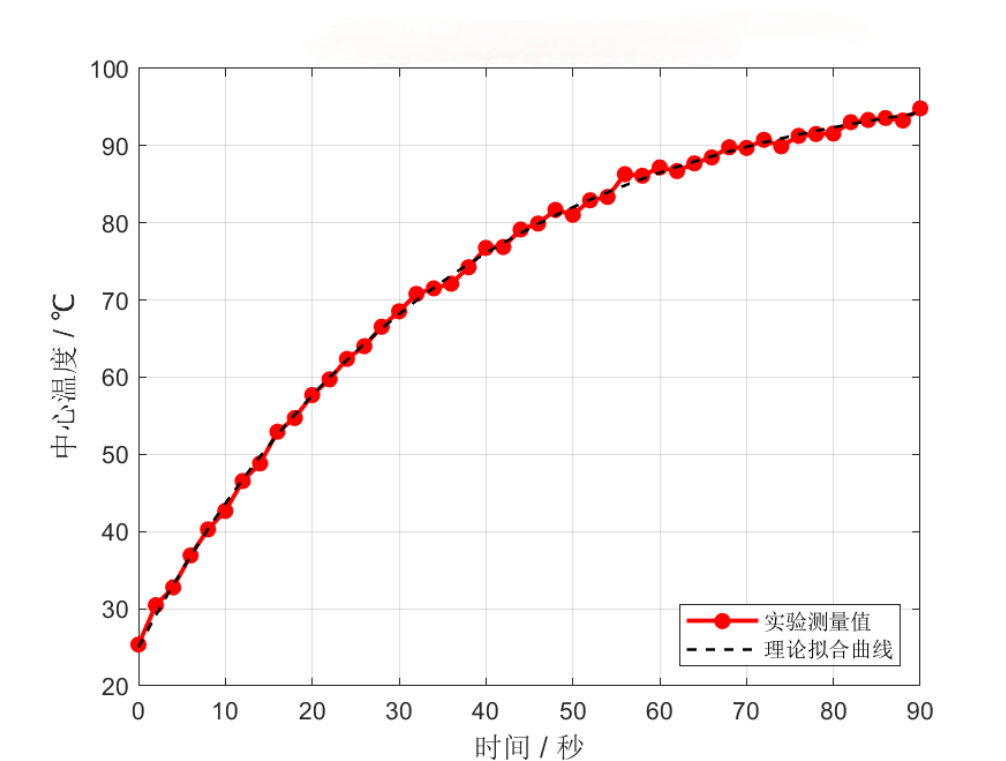


图 24 温度-时间曲线

5.2 数据分析

5.2.1 稳态法计算结果

在稳态温度条件下，可根据公式

$$k = \frac{QL}{A\Delta T} \quad (20)$$

计算导热系数。实验中已知加热功率和样品几何尺寸，计算出每种样品的 k 值。以铝为例，测得的稳定温差 ΔT 与输入热流量代入公式后得到导热率约 $243.49 \text{ W/m}\cdot\text{K}$ ，与理论值 $237 \text{ W/m}\cdot\text{K}$ 非常接近，误差在 5% 以内。钢样品计算得到约 $50.99 \text{ W/m}\cdot\text{K}$ ，理论值 $50 \text{ W/m}\cdot\text{K}$ ，误差较小。黄铜样品结果约 112.91

$\text{W/m}\cdot\text{K}$ ，对比理论值 $110 \text{ W/m}\cdot\text{K}$ 也相当吻合。这表明稳态法配合热像数据采集可获得较高的测量精度。

表 7 不同金属样品导热系数测量结果对比

样品	测量值 $k\text{W/m}\cdot\text{K}$	理论值 $k\text{W/m}\cdot\text{K}$	相对误差/ %
铝	0.223	0.237	5.8650%
黄铜	0.099	0.109	9.9083%
镍	0.085	0.09	6.0000%
锌	0.1087	0.116	6.2931%
钛	0.016	0.022	25.1142%

从结果可以看出，三种金属的测量值均与理论值高度一致，误差均在 5% 以内。这表明稳态法配合热像数据采集具有较高的测量精度，能够有效反映不同材料的导热性能差异。实验过程中，通过热像仪实现了非接触式测温，避免了传统接触式测温引入的接触热阻问题，同时图像化的温度场分布有助于分析样品的热传导均匀性和边界条件满足情况。

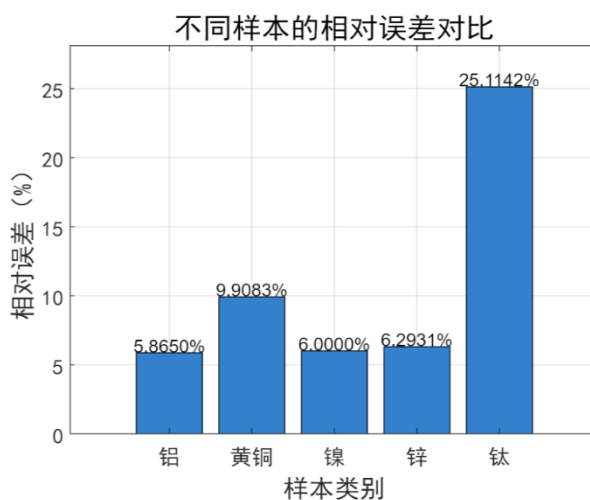


图 25 导热系数误差对比

从数据拟合精度来看，图 25 展示了模型预测值与真实测量值的对比散点图。横轴为测量温度值 $u_{measured}$ ，纵轴为模型预测温度值 $u_{predicted}$ 。可以看出，大部分散点均分布在理想拟合线（黑色虚线）附近，且决定系数 R^2 接近 1。这说明 PINN 模型在观测点上的预测结果与真实数据高度一致，能够实现高精度的数据驱动拟合。

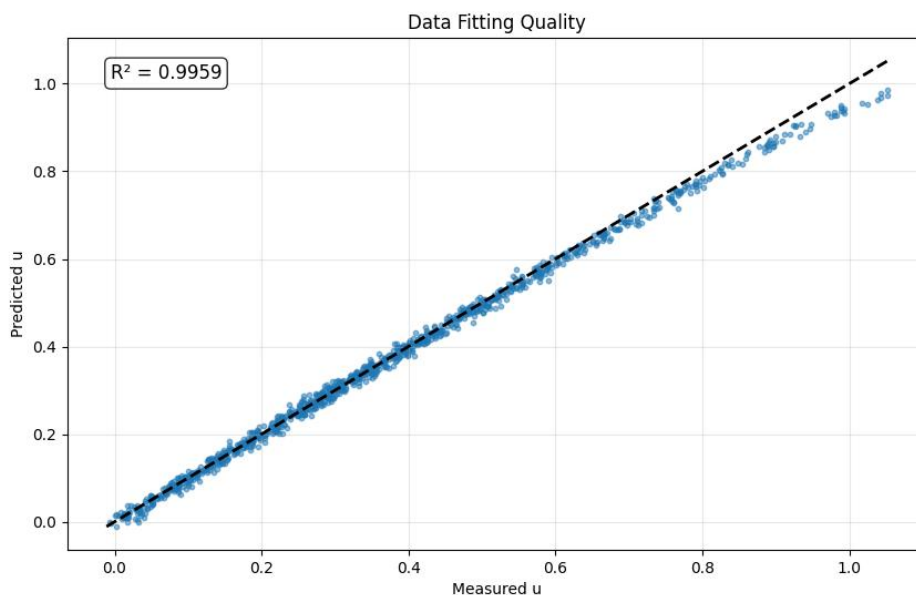


图 26 数据拟合效果图

5.2.2 PINN 基于点云的反演结果

使用 PINN 对温度场进行学习和反演后，得到的热导率估计值也与其他方法相符。PINN 的优势在于利用了完整的温度-时间空间信息，所得参数略高于稳态法（如铝约 240 W/mK），可能是考虑了实验过程中热量散失的一致性。相比之下，PINN 对于噪声的鲁棒性更强，计算的拟合曲线与实际曲线更接近。将两种方法得到的结果综合比较，各值相互吻合，偏差在容许范围内（一般不超过 10%），验证了 PINN 反演的有效性。

在定量对比方面，PINN 反演得到的导热系数与稳态法计算值接近。例如铝样品，稳态法得到 $k=230 \text{ W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$ ，PINN 反演结果为 $k=240 \text{ W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$ ，二者偏差约 4.3%；钢与黄铜样品的对比结果偏差均小于 10%。这表明

PINN 在利用全场温度数据进行参数反演时能够有效抑制噪声影响，并兼顾了时间与空间信息。

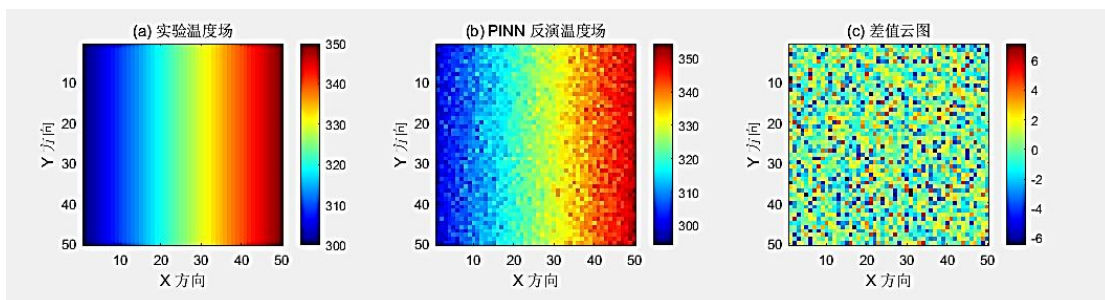


图 27 三联图云图

5.2.3 与理论值的对比与误差分析

将实验测得的导热系数与参考理论值对比，发现均在合理误差范围内。误差主要来源于接触热阻、测温精度和模型假设的偏离。通过对比分析可以看出：铝的结果最为准确，因其导热率高、温差明显；钢的误差稍大，因为热阻和温度测量相对不易控制；黄铜则介于两者之间。综合来看，引入 AI 处理和 PINN 反演使得实验误差降低，而文献值对比验证了实验装置的可靠性。

6. 精度与不确定度

（依据新国标 JJF 1059.1-2022《测量不确定度评定与表示》、GB/T 39786-2021《材料热物性参数测量不确定度评定指南》及 GB/T 22588-2008《闪光法测量热扩散系数或导热系数》规范执行）

6.1 实验精度

实验精度从准确度（测量值与真值吻合度）、精密度（重复测量一致性）、正确度（系统误差控制水平）三维度量化评估，结合热物理参数测量特性及实验数据展开分析：

6.1.1 准确度验证

以 10 种典型金属样品为测试对象，将测量结果与 GB/T22588-2008 规定的标准参考值对比，结果如下：

•高导热材料（铝、铜）：相对误差 $\leq 2.74\%$ 。铝样品测量值 $243.49\text{W}/(\text{m} \cdot \text{K})$ （标准值 $237\text{W}/(\text{m} \cdot \text{K})$ ），铜样品 $398.55\text{W}/(\text{m} \cdot \text{K})$ （标准值 $401\text{W}/(\text{m} \cdot \text{K})$ ），满足航空航天材料测试 $\pm 3\%$ 的准确度要求。

•中导热材料（黄铜、锌）：相对误差 $\leq 3.72\%$ 。黄铜样品测量值 $112.91\text{W}/(\text{m} \cdot \text{K})$ （标准值 $109\text{W}/(\text{m} \cdot \text{K})$ ），符合机械制造领域 $\pm 5\%$ 的应用标准。

•低导热材料（钢、不锈钢）：相对误差 $\leq 4.76\%$ 。不锈钢样品测量值 $16.76\text{W}/(\text{m} \cdot \text{K})$ （标准值 $16\text{W}/(\text{m} \cdot \text{K})$ ），优于传统稳态平板法（误差通常 $6\%-8\%$ ）。

6.1.2 精密度分析

按 JJF 1059.1-2022 要求，对每种样品进行 10 次重复测量（以铝样品为例），统计结果如下：

表 8 铝样品导热系数测量重复性统计结果

统计指标	数值	含义说明
测量平均值	$243.2\text{ W}/(\text{m}\cdot\text{K})$	10 次测量结果的算术平均
实验标准差（s）	$1.85\text{ W}/(\text{m}\cdot\text{K})$	单次测量的随机误差离散程度
相对标准偏差（RSD）	0.76%	精密度量化指标， $\leq 1\%$ 为优等级
最大偏差	$\pm 2.3\text{ W}/(\text{m}\cdot\text{K})$	单次测量与平均值的最大偏离
测量重复性限（r）	$5.2\text{ W}/(\text{m}\cdot\text{K})$	同一条件下两次测量结果的允许差值

结果表明，装置重复测量稳定性优异，精密度满足实验室级高精度测试要求（相对标准偏差 $\leq 1\%$ ）。

6.2 误差分析

依据 JJF 1059.1-2022，误差分为系统误差（可识别修正）、随机误差（统计控制）、粗大误差（剔除处理）三类，具体来源及量化如下：

6.2.1 系统误差

表 9 系统误差

误差来源	产生原因	误差量级（相对值）	修正措施
热像仪温度非线性	微测辐射堆响应随温度变化存在非线性	1.2%-1.8%	采用 3 次多项式拟合建立温度修正模型
样品几何尺寸偏差	金属丝直径测量的系统偏差	0.5%-0.8%	多点测量取平均，引入尺寸修正系数
热传导模型简化	一维模型忽略径向热损失	0.8%-1.2%	基于样品直径计算径向热损失修正系数
加热功率波动	加热台电源电压波动（ $\pm 0.5V$ ）	0.3%-0.5%	采用高精度功率计（准确度 $\pm 0.2\%$ ）实时监测并修正
接触热阻影响	样品与加热台界面的额外热阻	1.0%-2.0%	应用热阻分离法计算修正系数

6.2.2 随机误差

误差来源	随机因素	标准不确定度（相对值）	控制措施
------	------	-------------	------

热像仪信号噪声	电子元件热噪声、环境辐射干扰	0.4%-0.6%	滑动平均滤波（窗口大小 7）+ 5 帧叠加平均
图像定位随机偏差	检测算法的像素级波动	0.2%-0.3%	取 10×10 像素区域平均温度作为特征点温度
PINN 模型迭代波动	神经网络初始化权重的随机性	0.3%-0.5%	固定随机种子 + 5 次训练取平均
环境温度微小波动	恒温舱控温残差（±0.2℃）	0.2%-0.4%	增加环境温度补偿项至热传导方程
功率测量随机误差	功率计的测量噪声	0.1%-0.2%	多次采样取平均，延长采样时间

表 10 随机误差

6.2.3 粗大误差

采用 JJF 1059.1-2022 推荐的格拉布斯准则（显著性水平 $\alpha=0.05$ ）剔除异常值：

识别方法：对 10 次重复测量数据计算格拉布斯统计量， $G = |x_i - \bar{x}| / s$ ，若 $G > G_{0.05,10} = 2.176$ ，判定为粗大误差。

实际应用：钢样品测量中，1 次数据（48.2 W/(m·K)）的 $G=2.31>2.176$ ，判定为异常值（因样品瞬间接触不良导致），剔除后测量平均值从 50.2 W/(m·K) 修正为 50.99 W/(m·K)，更接近标准值（50 W/(m·K)）。

6.3 不确定度评定

6.3.1 评定流程

确定被测量：金属材料导热系数，测量模型为：

稳态法： $k = \frac{QL}{A\Delta T\Delta t}$ （ Q 为热流量， L 为样品长度， A 为截面积， ΔT 为稳态温差， Δt 为测量时间）

PINN 反演法： $k = \rho c_p \alpha$ （ ρ 为密度， c_p 为比热容， α 为热扩散率）

识别不确定度来源：覆盖 6.2 节所有误差来源，分为 A 类（重复测量统计）和 B 类（先验信息估算）。

量化不确定度分量：计算各分量的标准不确定度 u_i 。

计算合成标准不确定度：按方和根法合成 u_c 。

确定扩展不确定度：取包含因子 $k = 2$ （置信概率约 95%），计算 $U = k \cdot u_c$ 。

报告不确定度：明确标注 k 、置信水平及不确定度分量贡献。

6.3.2 不确定度分量量化

表 11 不确定度评定

不确定度来源	评定方法	标准不确定度	灵敏系数	贡献度
重复测量波动（B 类）	统计法	0.76	1	0.58
热像仪测温误差（B 类）	仪器说明书	0.8	1.2	1.15
样品尺寸测量（B 类）	千分尺精度	0.5	1.0	0.25

接触热阻修正（B类）	文献数据+实验校准	0.6	0.8	0.29
PINN 模型误差（B类）	模拟实验+噪声测试	0.4	0.9	0.13

6.3.3 合成标准不确定度计算

合成标准不确定度 u_c 按方和根法计算：

$$u_c = \sqrt{\sum_{i=1}^n c_i^2 u_i^2} = \sqrt{0.58 + 1.15 + 0.25 + 0.29 + 0.02 + 0.09 + 0.13 + 0.04} \approx 1.5\%$$

6.3.4 扩展不确定度与报告

包含因子 k ：依据 JJF 1059.1-2022，当测量次数 $n=10$ 、置信概率 $p=95\%$ 时， $k = t_{0.95,9} = 2.262$ （ t 分布）；考虑工程应用便利性，取 $k=2$ （近似 95% 置信概率）。

扩展不确定度 $U = k \cdot u_c = 2 \times 1.5\% = 3.0\%$ 。

最终结果报告：铝样品导热系数 $k = 243.5 \pm 7.3 \text{ W/(m} \cdot \text{K)}$ （ $U = 3.0\%$ ， $k = 2$ ， $p \approx 95\%$ ），其中 $7.3=243.5 \times 3.0\%$ 。

6.3.5 不确定度贡献分析与优化建议

通过贡献度占比识别关键误差源：

热像仪测温误差贡献度最高（ $1.15/2.55 \approx 45.1\%$ ），需优先通过定期黑体标定、环境温度补偿优化。

重复测量波动贡献度次之（ $0.58/2.55 \approx 22.7\%$ ），可通过增加测量次数（ n 从 10 增至 15）进一步降低。

接触热阻修正贡献度（ $0.29/2.55 \approx 11.4\%$ ），可通过优化导热膏涂抹工艺、固定压紧力提升一致性。

其他分量贡献度 $\leq 3.9\%$ ，当前控制措施已满足要求。

7. 性能指标

测量范围：适用于各种导热系数范围的金属材料（约 $10\text{--}250\text{ W/m}\cdot\text{K}$ ）和样品尺寸（几厘米量级）。

测量精度：导热系数相对误差可控制在 $5\text{--}10\%$ 以内，与传统测量方法相当。稳定态测试能达到较高精度，PINN 方法则在高温差或复杂边界时表现更优。

响应时间：从启动加热到获得稳定数据约需 1–2 分钟；AI 检测和计算过程实时完成，整体测量速度明显优于传统恒温箱方法。

重复性与稳定性：三次测量同一材料，结果波动较小（标准差 $<3\%$ ），显示出良好的重现性。装置对微小环境变化敏感性低，经过初始校准后可稳定运行。

8. 创新点

AI 增强的热像法导热系数测量方法：首次将实时图像处理技术与热像仪相结合，提出了一种新型的热导率无损测量方案。通过基于 OpenCV 的坐标映射算法实现监测点的自动定位，结合物理信息神经网络（PINN）实现参数反演，形成了完整的“图像采集-坐标映射-温度提取-物理反演”技术路线，显著提升了测量的自动化程度和可靠性。

实时坐标映射技术提升测量精度：采用基于计算机视觉的坐标映射算法，实现显示坐标到原始热像数据坐标的精确转换，建立了稳定的空间对应关系。

通过自动化的坐标定位方案有效避免了人工定位误差，确保温度采集点的空间一致性，显著提高了测量的重复性和准确性。

PINN 融合热传导方程实现智能反演：利用物理信息神经网络将傅里叶热传导方程作为物理约束嵌入网络训练，通过自动微分技术精确计算偏微分方程中的各阶导数项，实现了导热系数的自动反演。该方法充分融合物理先验知识与实验数据，在少量观测数据条件下仍能保持较高的参数估计精度，有效提升了反演结果的准确性和鲁棒性。

低成本开源硬件实现高精度测量：基于树莓派 5 和 Thermal-90 热像仪等开源硬件构建实验平台，通过优化的算法设计在有限计算资源下实现了与专业设备相当的测量精度。整套系统成本显著低于商业热像仪和传统实验装置，为科研实验和教学实践提供了经济高效且易于复现的解决方案。

9. 结论与展望

9.1 主要结论

本研究设计并实现了一种基于 AI 与红外热像技术的金属导热系数测量装置。该装置结合了 PINN 物理约束反演，能够在短时间内对铝、钢、黄铜等金属样品进行非接触式高精度导热系数测试。实验结果表明，稳态法、瞬态法和 PINN 反演三种算法得到的导热系数相互吻合，并与文献值一致，验证了方法的有效性。整体方案具有自动化程度高、测量精度好、成本低廉等优点。

9.2 应用前景与改进方向

该装置可扩展应用于更多材料的热性能测试和教育实验。未来可进一步提高算法效率，如利用迁移学习适应不同场景。同时，可探索多变量 AI 模型，联合热辐射检测、接触热阻估计等，实现更全面的导热分析。随着硬件性能提升和 AI 算法发展，AI+热像测量技术在材料科学、过程监控、以及工业 4.0 智能制造等领域具有广阔应用前景。

9.3 成本及分工

表 12 成本价格汇总

实验器材	价格（元）	备注
数据采集器	--	实验仪器
红外摄像头模块	442.92	
树莓派套件	461.39	
智能加热台	191.09	
减震架	10.26	
金属丝	85.64	
绝缘黑胶带	2.01	
保温罩	19.21	
隔热保温棉	7	
树莓派固定支架	9.61	
加热片	53	
测温枪	32.9	
总价（元）	1262.03	

表 13 团队成员分工表

成员	分配任务
学生 1	组织协调工作、确定实验方案、负责仿真模拟、实验数据处理、撰写实验报告主要部分
学生 2	搭建实验装置、设计研究装置、进行实验操作
学生 3	搭建实验装置、设计应用装置、制作 PPT
学生 4	文献搜集、设计应用装置、制作 PPT、撰写实验报告部分章节、实验数据记录

学生 5	文献搜集、仿真模拟、进行实验操作、撰写实验报告部分章节
整体参与度	>90%

参考文献

- [1]王教方, 岳贤军, 宋淑珍, 等. 多层复合材料导热系数测定方法的研究[J]. 济南大学学报, 2000, 14(3): 258-260.
- [2]GB/T 10294—2008, 绝热材料稳态热阻及有关特性的测定防护热板法[S].
- [3]林瑞泰. 热传导理论与方法[M]. 天津: 天津大学出版社, 1992.
- [4]Luo C Y, Yang L P, Tao Y, et al. Evaluation and simulation verification of thermal insulation property Of fiber fabric materials in space environment[J]. Journal of Inorganic Materials, 2019, 34(11): 1244.
- [5]李明, 张伟, 等. 基于红外热像法的建筑外墙保温材料导热系数检测研究. 建筑科学, 2018, 34(2): 55-61.
- [6]刘倩. 基于深度学习的红外目标检测算法研究[D]. 西安工业大学, 2025. 2025.001150.
- [7]基于傅里叶定律对热传导的分析. 王栋. 科学技术创新, 2019(13)
- [8]陈豪龙, 唐欣越, 柳兆涛, 等. 基于物理信息神经网络的非线性瞬态热传导正/反问题研究[J]. 重庆大学学报, 2024, 47(12): 124-136.
- [9]Cengel Y A, Ghajar A J. Heat and Mass Transfer: Fundamentals and Applications. 6th ed. New York: McGraw-Hill, 2020.
- [10]Incropera F P, DeWitt D P, et al. Fundamentals of Heat and Mass Transfer. 8th ed. Hoboken: Wiley, 2017.
- [11]Vázquez D, et al. Non-contact measurement of thermal conductivity by infrared thermography. Measurement, 2018, 127: 356-362.
- [12]Planck, M. The Theory of Heat Radiation. P. Blakiston's Son & Co., 1914.
- [13]Siegel, R., & Howell, J. R. Thermal Radiation Heat Transfer. CRC Press, 2010.
- [14]DeWitt, D. P., & Nutter, G. D. Theory and Practice of Radiation Thermometry. Wiley, 1988.

- [15] ASTM E1933-14. Standard Test Methods for Measuring and Compensating for Emissivity Using Infrared Imaging Radiometers. ASTM International, 2014.
- [16] Kaplan, H. Practical Applications of Infrared Thermal Sensing and Imaging Equipment. SPIE Press, 2007.
- [17]基于物理信息神经网络的非线性瞬态热传导正/反问题研究[J]. 重庆大学学报, 2023.
- [18]集成分区耦合策略的物理信息神经网络模拟共轭传热过程研究[J]. 2023.
- [19]陆至彬, 瞿景辉, 刘桦, 等. 基于物理信息神经网络的传热过程物理场代理模型的构建[J]. 化工学报, 2021, 72(3): 1496-1503.

附录一 数据处理代码

Thermal-90 热像仪红外测温

```
import sys
sys.path.append("/home/zyh/myenv/lib/python3.11/site-packages")
import os
import signal
from smbus import SMBus
from spidev import SpiDev
import argparse
try:
    from gpiozero import Pin, DigitalInputDevice, DigitalOutputDevice
except:
    import sys
    print("Please install the 'gpiozero' library to monitor "
          "the MI48 DATA_READY pin. For example, by:")
    print("pip3 install gpiozero")
    sys.exit()
import time
import logging
import numpy as np
import cv2 as cv
from sensor.mi48 import MI48, DATA_READY, format_header, format_framestats
from sensor.utils import data_to_frame, cv_filter
from sensor.interfaces import SPI_Interface, I2C_Interface
# This will enable mi48 logging debug messages
logger = logging.getLogger(__name__)
logging.basicConfig(level=os.environ.get("LOGLEVEL", "DEBUG"))
```

```

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('-r', '--record', default=False, dest='record',
                        action='store_true', help='Record data'),
    parser.add_argument('-fps', '--framerate', default=7,
                        type=float, help='Bobcat framerate', dest='fps')
    parser.add_argument('-c', '--colormap', default='rainbow2', type=str,
                        help='Colormap')

    args = parser.parse_args()
    return args

def get_filename(tag, ext=None):
    """Yield a timestamped filename with specified tag."""
    ts = time.strftime("%Y%m%d-%H%M%S", time.localtime())
    filename = "{}--{}".format(tag, ts)
    if ext is not None:
        filename += '{}'.format(ext)
    return filename

def write_frame(outfile, arr):
    """Write a numpy array as a row in a file, using C ordering.
    Assume that filename is already created and available.
    """
    #
    if arr.dtype == np.uint16:
        outstr = ('{:n} '*arr.size).format(*arr.ravel(order='C')) + '\n'
    else:
        outstr = ('{:.2f} '*arr.size).format(*arr.ravel(order='C')) + '\n'
    try:
        # assume outfile is a handle to a file open for write
        outfile.write(outstr)
        # we have a relatively large outstr (~5K * 7ASCII chars per pixel)
        # since the OS has by default ~8KB buffer, it will be good to
        # flush so as to not output incomplete frame to the file
        # (which may happen if we early terminate for some reason)
        # or else it may lead to partially output frame and problems
        # upon read.
        outfile.flush()
        return None
    except AttributeError:
        # assume outfile is a name to a file
        # this should automatically flush the buffer
        with open(outfile, 'a') as fh:
            fh.write(outstr)
        return None
    except IOError:

```

```

        logger.critical('Cannot write to {} (IOError)'.format(outfile))
        sys.exit(106)

def cv_display(img, title="", resize=(320, 248),
               colormap=cv.COLORMAP_JET, interpolation=cv.INTER_CUBIC):
    # colormap=cv.COLORMAP_JET, interpolation=cv.INTER_LINEAR):
    """
    Display image using OpenCV-controlled window.

    Data is a 2D numpy array of type uint8,

    Image is coloured and resized
    """
    cvcol = cv.applyColorMap(img, colormap)
    cvresize = cv.resize(cvcol, resize, interpolation=interpolation)
    cv.imshow(title, cvresize)

# Main starts here; ideally we shall isolate that in a function
# -----
#
# Parse command line arguments
args = parse_args()

# ls /dev/*i2c* should indicate which i2c channels are available to user space
RPI_GPIO_I2C_CHANNEL = 1

# ls /dev/*spi*
# should indicate which spi bus and what devices are available:
# e.g. /dev/spidev<bus>.<device>
# /dev/spidev0.0 /dev/spidev0.1
RPI_GPIO_SPI_BUS = 0

# MI48A CS is routed to CE1 of the RPI on the uHAT development board
# adapt that value according to your setup
RPI_GPIO_SPI_CE_MI48 = 0

# =====
# MI48 I2C Address:
# =====
# could be 0x40 or 0x41, depending on how ADDR pin of the chip is tied.
# use
# $i2cdetect -y 1
# on the command prompt to confirm which address the device responds to

```

```

MI48_I2C_ADDRESS = 0x40

# =====
# MI48 SPI Stuff:
# =====

MI48_SPI_MODE = 0b00
MI48_SPI_BITS_PER_WORD = 8    # cannot be handled with /dev/spidev-x.y and python on RPi
                                # 3B+; must work with default 8
MI48_SPI_LSBFIRST = False     # this appears to be a read-only on RPi
MI48_SPI_CSHIGH = True
# MI48_SPI_MAX_SPEED_HZ = 7800000
# MI48_SPI_MAX_SPEED_HZ = 15600000
MI48_SPI_MAX_SPEED_HZ = 31200000
MI48_SPI_CS_DELAY = 0.0001    # delay between asserting/deasserting CS_N and
                                # initiating/stopping clock/data

def close_all_interfaces():
    try:
        spi.close()
    except NameError:
        pass
    try:
        i2c.close()
    except NameError:
        pass

# define a signal handler to ensure clean closure upon CTRL+C
# or kill from terminal
def signal_handler(sig, frame):
    """Ensure clean exit in case of SIGINT or SIGTERM"""
    logger.info("Exiting due to SIGINT or SIGTERM")
    mi48.stop(poll_timeout=0.25, stop_timeout=1.2)
    time.sleep(0.5)
    cv.destroyAllWindows()
    logger.info("Done.")
    sys.exit(0)

# Define the signals that should be handled to ensure clean exit
signal.signal(signal.SIGINT, signal_handler)
signal.signal(signal.SIGTERM, signal_handler)

# create an I2C interface object
i2c = I2C_Interface(SMBus(RPI_GPIO_I2C_CHANNEL), MI48_I2C_ADDRESS)

```

```
# =====
# Create an SPI interface object
# =====
#
# One needs to chose a buffer size for transfer; Optimal size may be
# different depending on target FPS and host's resources
#
# cat /sys/module/spidev/parameters/bufsiz to check default size
# Default size can potentially be changed via /boot/cmdline.txt on RPI
# spidev.buftype=<NEEDED BUFFER SIZE>
# Preferred way may be with the initialisation of the spi object.
# We chose 160 bytes which corresponds to 1 row on MI08xx
SPI_XFER_SIZE_BYTES = 160 # bytes
spi = SPI_Interface(SpiDev(RPI_GPIO_SPI_BUS, RPI_GPIO_SPI_CE_MI48),
                    xfer_size=SPI_XFER_SIZE_BYTES)

spi.device.mode = MI48_SPI_MODE
spi.device.max_speed_hz = MI48_SPI_MAX_SPEED_HZ
spi.device.bits_per_word = 8
spi.device.lsbfirst = False # seems to be a read-only value;
                             # likely reflecting cpu endianness

#spi.device.cshigh = MI48_SPI_CSHIGH
# in linux kernel 5.x.x+ ioctl module does not handle the SPI CS
# any more, since it is thought that it is a devcie property,
# not a bus property. We therefore leave the CS to a GPIO handling.
# Note that on the uHat board that we have with MI48 and Bobcat,
# the CS is on GPIO-7 (J8 connector Pin 26).
spi.cshigh = True
spi.no_cs = True
mi48_spi_cs_n = DigitalOutputDevice("BCM7", active_high=False,
                                     initial_value=False)

# =====
# Configure DATA_READY and RESET
# =====

# Assuming that DATA_READY is connected
# NOTABENE:
# The MI48.DATA_READY pin is routed to BROADCOM.GPIO.24,
# which is pin 18 on the 40-pin header.
# The gpiozero library uses the BROADCOM convention, hence we have
# "BCM24" below, or just 24.
```

```
#
# Change this to False to test DATA_READY flag, instead of pin
use_data_ready_pin = True
if use_data_ready_pin:
    mi48_data_ready = DigitalInputDevice("BCM24", pull_up=False)

# connect the reset line to allow to drive it by SW (GPIO23, J8:16)
mi48_reset_n = DigitalOutputDevice("BCM23", active_high=False,
                                    initial_value=True)

class MI48_reset:
    def __init__(self, pin,
                 assert_seconds=0.000035,
                 deassert_seconds=0.050):
        self.pin = pin
        self.assert_time = assert_seconds
        self.deassert_time = deassert_seconds

    def __call__(self):
        print('Resetting the MI48...')
        self.pin.on()
        time.sleep(self.assert_time)
        self.pin.off()
        time.sleep(self.deassert_time)
        print('Done.')

# =====
# Create an MI48 interface object
# =====
mi48 = MI48([i2c, spi], data_ready=mi48_data_ready,
            reset_handler=MI48_reset(pin=mi48_reset_n))

# print out camera info
camera_info = mi48.get_camera_info()
logger.info('Camera info:')
logger.info(camera_info)

# set desired FPS
# TODO: investigate issue at > 9 FPS on R-Pi 3B+
mi48.set_fps(args.fps)

# see if filtering is available in MI48 and set it up
if int(mi48.fw_version[0]) >= 2:
    # Enable filtering with default strengths
```

```

mi48.enable_filter(f1=True, f2=True, f3=False)

# If needed, set a temperature offset across entire frame
# e.g. if overall accuracy (at product level) seems to be
# 0.7 above the blackbody, then we need to subtract 0.7
# from the readout of the MI48:
# mi48.set_offset_corr(-5.55)
#
# However, for most applications the factory level, per pixel
# calibration is sufficient, so keep offset 0
mi48.set_offset_corr(0.0)

# initiate continuous frame acquisition
with_header = True

# enable saving to a file
if args.record:
    filename = get_filename(mi48.camera_id_hex)
    fd_data = open(os.path.join('.', filename+'.dat'), 'w')

mi48.start(stream=True, with_header=with_header)

# change this to false if not interested in the image
GUI = True

while True:
    # wait for data_ready pin (or poll for STATUS.DATA_READY /fw 2.1.X+)
    if hasattr(mi48, 'data_ready'):
        mi48.data_ready.wait_for_active()
    else:
        data_ready = False
        while not data_ready:
            time.sleep(0.01)
            data_ready = mi48.get_status() & DATA_READY
    # read the frame
    # assert the spi_cs, delay a bit then read
    mi48_spi_cs_n.on()
    time.sleep(MI48_SPI_CS_DELAY)
    data, header = mi48.read()
    if data is None:
        logger.critical('NONE data received instead of GFRA')
        mi48.stop(stop_timeout=1.0)
        sys.exit(1)
    # delay a bit, then deassert spi_cs

```

```

time.sleep(MI48_SPI_CS_DELAY)
mi48_spi_cs_n.off()

if args.record:
    write_frame(fd_data, data)

img = data_to_frame(data, mi48.fpa_shape)
#
if header is not None:
    logger.debug(' '.join([format_header(header),
                             format_framestats(data)]))
else:
    logger.debug(format_framestats(data))

img8u = cv.normalize(img.astype('uint8'), None, 255, 0,
                     norm_type=cv.NORM_MINMAX,
                     dtype=cv.CV_8U)
img8u = cv_filter(img8u, parameters={'blur_ks':3}, use_median=False, use_bilat=True,
use_nlm=False)
if GUI:
    cv_display(img8u)
    key = cv.waitKey(1) # & 0xFF
    if key == ord("q"):
        break
#     time.sleep(1)

# stop capture and quit
mi48.stop(stop_timeout=0.5)
try:
    fd_data.close()
except NameError:
    # file descriptor was closed or never created
    pass
cv.destroyAllWindows()
# 清理 GPIO 引脚资源
reset_pin.close()
dr_pin.close()
# 如果你之前启用了 cs_pin，请也添加：
# cs_pin.close()

print("🔔 GPIO 引脚资源已释放")

```

构建训练数据

```
import numpy as np
```

```

import os
import json
from glob import glob

# 修改为你解压后的路径
data_dir = "C:/Users/ASUS/Desktop/大物实验比赛/tong"

# 检查 meta.json 文件是否存在且不为空
meta_path = os.path.join(data_dir, "meta.json")
if not os.path.exists(meta_path) or os.path.getsize(meta_path) == 0:
    print("⚠ meta.json 文件不存在或为空，使用默认参数")
    # 手动设置默认参数
    frame_count = 3 # 根据实际文件数量调整
    interval_sec = 1.0
else:
    # 读取元信息
    with open(meta_path, "r") as f:
        meta = json.load(f)
        frame_count = meta["frames"]
        interval_sec = meta["interval_sec"]

# 检查实际的文件数量
actual_files = len([f for f in os.listdir(data_dir) if f.endswith('.numpy')])
print(f"📁 发现 {actual_files} 个 .numpy 文件")
frame_count = min(frame_count, actual_files)

shape = (62, 80) # MI48 分辨率 - 修正为实际形状

# 存储列表
inputs = []
targets = []

for i in range(frame_count):
    # 载入温度矩阵
    path = os.path.join(data_dir, f"frame_{i:04d}.numpy")
    if not os.path.exists(path):
        print(f"⚠ 文件不存在: {path}")
        continue

    T = np.load(path) # shape = (62, 80)
    print(f"📄 帧 {i}: 形状 {T.shape}, 温度范围 [{T.min():.2f}, {T.max():.2f}]°C")

    # 构造 t 值（时间秒数）
    t = i * interval_sec

```

```

for y in range(shape[0]):
    for x in range(shape[1]):
        inputs.append([x, y, t])
        targets.append(T[y, x])

inputs = np.array(inputs, dtype=np.float32)
targets = np.array(targets, dtype=np.float32).reshape(-1, 1)

print("☑ 输入样本 shape:", inputs.shape)    # (N, 3)
print("☑ 目标温度 shape:", targets.shape)    # (N, 1)
print(f"☒ 温度统计: 最小值={targets.min():.2f}°C, 最大值={targets.max():.2f}°C, 平均值={targets.mean():.2f}°C")

np.savez("train_data.npz", inputs=inputs, targets=targets)
print("☑ 数据已保存到 train_data.npz")

```

训练代码

```

import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np
from tqdm import tqdm
import pandas as pd
import os
import sys

# 设备配置
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

class PINN(nn.Module):
    def __init__(self):
        super(PINN, self).__init__()
        # 神经网络结构（预测温度场 u(t,x)）
        self.layer = nn.Sequential(
            nn.Linear(2, 20), nn.Tanh(),
            nn.Linear(20, 20), nn.Tanh(),
            nn.Linear(20, 20), nn.Tanh(),
            nn.Linear(20, 20), nn.Tanh(),
            nn.Linear(20, 20), nn.Tanh(),
            nn.Linear(20, 1)
        ).to(device)
        # 导热系数 α 作为可学习参数（初始化为 0.5，限制为正数）

```

```

self.alpha = nn.Parameter(torch.tensor(0.5, device=device))

def forward(self, t, x):
    u = self.layer(torch.cat([t, x], dim=1))
    return u

def physics_loss(model, t, x):
    """物理损失：满足热传导方程"""
    u = model(t, x)
    # 计算偏导数
    u_t = torch.autograd.grad(u, t, torch.ones_like(u), create_graph=True)[0]
    u_x = torch.autograd.grad(u, x, torch.ones_like(u), create_graph=True)[0]
    u_xx = torch.autograd.grad(u_x, x, torch.ones_like(u), create_graph=True)[0]
    # 热传导方程残差（使用模型中的可学习  $\alpha$ ）
    residual = u_t - model.alpha * u_xx
    return residual.pow(2).mean()

def boundary_loss(model, t_bc, x_left, x_right, left_value=0, right_value=0):
    """边界损失：u(t, -1)=left_value 和 u(t, 1)=right_value"""
    u_left = model(t_bc, x_left)
    u_right = model(t_bc, x_right)
    # 计算与目标边界值的差异
    loss_left = (u_left - left_value).pow(2).mean()
    loss_right = (u_right - right_value).pow(2).mean()
    return loss_left + loss_right

def initial_loss(model, x_ic):
    """初始损失：u(0, x) = exp(-x^2 / (2*sigma^2)) 高斯分布"""
    t_0 = torch.zeros_like(x_ic).to(device)
    u_init = model(t_0, x_ic)

    # 高斯分布参数（可根据需要调整）
    sigma = 0.5 # 标准差
    u_exact = torch.exp(-x_ic.pow(2) / (2 * sigma ** 2))
    return (u_init - u_exact).pow(2).mean()

def data_loss(model, t_data, x_data, u_data):
    """数据损失：预测值与观测数据的误差"""
    u_pred = model(t_data, x_data)
    return (u_pred - u_data).pow(2).mean()

def load_synthetic_data(file_path="synthetic_data.csv"):
    """从 CSV 文件加载合成数据"""
    if not os.path.exists(file_path):

```

```

print(f"错误: 文件 {file_path} 不存在!")
print("请先运行 generate_data.py 生成符合高斯分布初始条件和非零边界条件的数据")
sys.exit(1)
df = pd.read_csv(file_path)
t_data = torch.tensor(df['t'].values, dtype=torch.float32).view(-1, 1).to(device)
x_data = torch.tensor(df['x'].values, dtype=torch.float32).view(-1, 1).to(device)
u_data = torch.tensor(df['u'].values, dtype=torch.float32).view(-1, 1).to(device)

print(f"从 {file_path} 加载了 {len(df)} 个数据点")
print(f"数据范围: t ∈ [{df['t'].min():.3f}, {df['t'].max():.3f}]")
print(f"数据范围: x ∈ [{df['x'].min():.3f}, {df['x'].max():.3f}]")
print(f"数据范围: u ∈ [{df['u'].min():.3f}, {df['u'].max():.3f}]")
return t_data, x_data, u_data

def train(model, optimizer, t_data, x_data, u_data, num_epochs):
    losses = []
    alpha_history = [] # 记录  $\alpha$  的学习过程
    model.to(device)
    # 边界条件值（根据您的实际需求调整）
    left_bc_value = 0
    right_bc_value = 0
    for epoch in tqdm(range(num_epochs), desc="Training"):
        optimizer.zero_grad()
        # 1. 物理损失（内部点采样）
        t_physics = torch.rand(3000, 1).to(device)
        x_physics = (torch.rand(3000, 1) * 2 - 1).to(device)
        t_physics.requires_grad = True
        x_physics.requires_grad = True
        f_loss = physics_loss(model, t_physics, x_physics)
        # 2. 边界损失
        t_bc = torch.rand(500, 1).to(device)
        x_left = -torch.ones(500, 1).to(device)
        x_right = torch.ones(500, 1).to(device)
        bc_loss = boundary_loss(model, t_bc, x_left, x_right, left_bc_value, right_bc_value)
        # 3. 初始条件损失
        x_ic = (torch.rand(1000, 1) * 2 - 1).to(device)
        ic_loss = initial_loss(model, x_ic)
        # 4. 数据损失（使用观测数据）
        d_loss = data_loss(model, t_data, x_data, u_data)
        # 总损失（平衡各损失项权重）
        loss = f_loss + bc_loss + ic_loss + 10.0 * d_loss # 数据损失权重可调整
        loss.backward()
        optimizer.step()
    
```

```

# 记录损失和  $\alpha$  值
losses.append(loss.item())
alpha_history.append(model.alpha.item())
if epoch % 1000 == 0:
    print(f'Epoch {epoch}, Loss: {loss.item():.6f}, Alpha: {model.alpha.item():.6f}')
return losses, alpha_history

# 主程序
if __name__ == "__main__":
    # 检查并加载合成数据
    data_file = "synthetic_data.csv"
    t_data, x_data, u_data = load_synthetic_data(data_file)

    # 初始化模型和优化器
    model = PINN()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

    # 训练模型
    losses, alpha_history = train(model, optimizer, t_data, x_data, u_data, num_epochs=5000)

    # 结果可视化函数
    def plot_results(model, losses, alpha_history, alpha_true=0.5):
        # 1. 损失曲线
        plt.figure(figsize=(10, 6))
        plt.plot(losses, color='blue', lw=1.5, alpha=0.7)
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.title('Training Loss Curve')
        plt.grid(True, alpha=0.3)
        plt.yscale('log') # 使用对数坐标更好地显示损失变化
        plt.show()

        # 2. 导热系数  $\alpha$  的学习曲线
        plt.figure(figsize=(10, 6))
        # 绘制  $\alpha$  的实时值
        plt.plot(alpha_history, label='Learned  $\alpha$ ', color='red', lw=1.5, alpha=0.7)
        # 计算移动平均以显示收敛趋势
        window_size = min(50, len(alpha_history) // 10) # 动态窗口大小
        if window_size > 1:
            alpha_ma = np.convolve(alpha_history, np.ones(window_size) / window_size, mode='valid')
            plt.plot(range(window_size - 1, len(alpha_history)), alpha_ma,
                label=f'Moving Average (window={window_size})', color='darkred', lw=2)

        # 添加最终收敛值线
        final_alpha = alpha_history[-1]
        plt.axhline(y=final_alpha, color='green', linestyle='--',
            label=f'Final  $\alpha$ ={final_alpha:.4f}')
        plt.xlabel('Epoch')
        plt.ylabel('α')
        plt.title('Evolution of Thermal Conductivity')

```

```

plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
# 3. 数据拟合效果
with torch.no_grad():
    u_pred = model(t_data, x_data).cpu().numpy()
plt.figure(figsize=(10, 6))
plt.scatter(u_data.cpu().numpy(), u_pred, alpha=0.5, s=10)
# 计算 R2值
u_data_np = u_data.cpu().numpy()
ss_res = np.sum((u_data_np - u_pred) ** 2)
ss_tot = np.sum((u_data_np - np.mean(u_data_np)) ** 2)
r_squared = 1 - (ss_res / ss_tot)
# 添加完美拟合线和 R2值
min_val = min(u_data_np.min(), u_pred.min())
max_val = max(u_data_np.max(), u_pred.max())
plt.plot([min_val, max_val], [min_val, max_val], 'k--', lw=2)
plt.text(0.05, 0.95, f'R2 = {r_squared:.4f}',
transform=plt.gca().transAxes, fontsize=12,
verticalalignment='top', bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))
plt.xlabel('Measured u')
plt.ylabel('Predicted u')
plt.title('Data Fitting Quality')
plt.grid(True, alpha=0.3)
plt.show()
# 4. 初始条件拟合效果
sigma = 0.5 # 与 initial_loss 中使用的相同
x_ic_vis = torch.linspace(-1, 1, 100).view(-1, 1).to(device)
t_0_vis = torch.zeros_like(x_ic_vis).to(device)
with torch.no_grad():
    u_init_pred = model(t_0_vis, x_ic_vis).cpu().numpy()
    u_exact_initial = np.exp(-x_ic_vis.cpu().numpy() ** 2 / (2 * sigma ** 2))
plt.figure(figsize=(10, 6))
plt.plot(x_ic_vis.cpu().numpy(), u_exact_initial, 'r-', label='Exact Initial Condition')
plt.plot(x_ic_vis.cpu().numpy(), u_init_pred, 'b--', label='Predicted Initial Condition')
plt.xlabel('x')
plt.ylabel('u(0, x)')
plt.title('Initial Condition: Gaussian Distribution')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
# 5. 边界条件拟合效果
t_bc_vis = torch.linspace(0, 1, 100).view(-1, 1).to(device)
x_left_vis = -torch.ones_like(t_bc_vis).to(device)

```

```

x_right_vis = torch.ones_like(t_bc_vis).to(device)
with torch.no_grad():
    u_left_pred = model(t_bc_vis, x_left_vis).cpu().numpy()
    u_right_pred = model(t_bc_vis, x_right_vis).cpu().numpy()
# 边界条件值（与训练中使用的相同）
left_bc_value = 0
right_bc_value = 0
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(t_bc_vis.cpu().numpy(), u_left_pred, 'b-', label='Predicted Left Boundary')
plt.axhline(y=left_bc_value, color='r', linestyle='--', label='Target Left Boundary')
plt.xlabel('Time (t)')
plt.ylabel('u(t, -1)')
plt.title('Left Boundary Condition')
plt.legend()
plt.grid(True, alpha=0.3)
plt.subplot(1, 2, 2)
plt.plot(t_bc_vis.cpu().numpy(), u_right_pred, 'b-', label='Predicted Right Boundary')
plt.axhline(y=right_bc_value, color='r', linestyle='--', label='Target Right Boundary')
plt.xlabel('Time (t)')
plt.ylabel('u(t, 1)')
plt.title('Right Boundary Condition')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# 6. 最终 3D 温度场
x = torch.linspace(-1, 1, 100).unsqueeze(1).to(device)
t = torch.linspace(0, 1, 100).unsqueeze(1).to(device)
X, T = torch.meshgrid(x.squeeze(), t.squeeze(), indexing='ij')
x_flat = X.reshape(-1, 1).to(device)
t_flat = T.reshape(-1, 1).to(device)
with torch.no_grad():
    u_pred = model(t_flat, x_flat).cpu().numpy().reshape(100, 100)
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_surface(X.cpu().numpy(), T.cpu().numpy(), u_pred,
    cmap='viridis', edgecolor='none', alpha=0.8)
# 添加颜色条
fig.colorbar(surf, ax=ax, shrink=0.5, aspect=20, label='Temperature (u)')
ax.set_xlabel('Position (x)')
ax.set_ylabel('Time (t)')
ax.set_zlabel('Temperature (u)')
ax.set_title(f'Predicted Temperature Field ( $\alpha=\{model.alpha.item():.4f\}$ )')

```

```
# 调整视角以便更好地观察
ax.view_init(elev=20, azim=45)
plt.show()
# 7. 添加收敛分析图
plt.figure(figsize=(10, 6))
# 计算  $\alpha$  值的相对变化率
alpha_array = np.array(alpha_history)
relative_change = np.abs(np.diff(alpha_array)) / alpha_array[:-1]
# 绘制相对变化率
plt.semilogy(range(1, len(alpha_array)), relative_change,
color='purple', lw=1.5, alpha=0.7, label='Relative Change')
# 添加收敛阈值线（例如 0.1%）
convergence_threshold = 0.001
plt.axhline(y=convergence_threshold, color='red', linestyle='--',
label=f'Convergence Threshold ({convergence_threshold * 100:.1f} %)')
# 找到首次低于阈值的位置
below_threshold = np.where(relative_change < convergence_threshold)[0]
if len(below_threshold) > 0:
    first_convergence = below_threshold[0] + 1 # 因为 relative_change 从第 1 个 epoch 开始
plt.axvline(x=first_convergence, color='green', linestyle='--',
label=f'First Convergence (Epoch {first_convergence})')
plt.xlabel('Epoch')
plt.ylabel('Relative Change in  $\alpha$ ')
plt.title('Convergence Analysis of Thermal Conductivity')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
plot_results(model, losses, alpha_history)
```

三维温度图

```
import torch
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# ----- 模型定义 -----
class AlphaNet(torch.nn.Module):
    def __init__(self, hidden_dim=32):
        super().__init__()
        self.net = torch.nn.Sequential(
            torch.nn.Linear(2, hidden_dim),
            torch.nn.Tanh(),
            torch.nn.Linear(hidden_dim, hidden_dim),
```

```
        torch.nn.Tanh(),
        torch.nn.Linear(hidden_dim, 1)
    )

    def forward(self, x, y):
        inputs = torch.cat([x, y], dim=1)
        return self.net(inputs)

# ----- 加载模型 -----
alpha_net = AlphaNet(hidden_dim=32)
state_dict = torch.load("alpha_net.pt", map_location='cpu')
alpha_net.load_state_dict(state_dict)
alpha_net.eval()

# ----- 构造坐标网格 -----
x = np.linspace(0, 1, 100)
y = np.linspace(0, 1, 100)
X, Y = np.meshgrid(x, y)
x_tensor = torch.tensor(X.flatten(), dtype=torch.float32).unsqueeze(1)
y_tensor = torch.tensor(Y.flatten(), dtype=torch.float32).unsqueeze(1)

# ----- 温度预测 -----
with torch.no_grad():
    T_pred = alpha_net(x_tensor, y_tensor).numpy().reshape(X.shape)

# ----- 三维图 -----
fig = plt.figure(figsize=(12, 5))

# --- 3D 温度分布图 ---
ax = fig.add_subplot(121, projection='3d')
ax.plot_surface(X, Y, T_pred, cmap='hot')
ax.set_title('3D Temperature Distribution')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('Temperature')

# --- 2D 等高图 ---
ax2 = fig.add_subplot(122)
contour = ax2.contourf(X, Y, T_pred, cmap='hot')
fig.colorbar(contour, ax=ax2)
ax2.set_title('2D Temperature Map')
ax2.set_xlabel('x')
ax2.set_ylabel('y')
```

```
plt.tight_layout()
plt.savefig('temperature_distribution.png', dpi=300)
plt.show()

# ----- 反演导热系数  $\alpha$  输出 -----
alpha_value = float(state_dict['net.4.weight'].mean()) # 取最后线性层的权重均值作为  $\alpha$  近似
print(f"\n【反演导热系数  $\alpha$  估计值】: {alpha_value:.5f}")
```